Table of Contents

1 · Dossier Guide and Completeness Audit 1

How to use this dossier . . . 1

The honest audit: what v1 did not contain . . . 2

The 360-degree completeness matrix . . . 4

Risk register: the things most likely to actually hurt . . . 6

2 · Product Requirements Document 8

Vision, persona, and jobs to be done . . . 8

Scope: MoSCoW . . . 9

Functional requirements - Core Loop . . . 9

Functional requirements - Deal Intelligence, Coaching, Library, Platform . . . 10

Non-functional requirements . . . 12

Out of scope, and the glossary . . . 14

3 · Functional Specification - The Four Pillars 15

The Core Loop, step by step . . . 15

The call state machine . . . 17

Deal Intelligence rules . . . 18

Coaching specification . . . 20

Library specification . . . 21

Edge cases and the error matrix . . . 21

4 · Data Dictionary and Schema Reference 23

Core entities: Deal, Contact, Call, MediaAsset . . . 23

Transcript entities: Transcript, Segment . . . 27

Intelligence entities: Insight, Scorecard, Note, FollowUpDraft, BestPractice . . . 28

AI engine entities: ProviderRun, PromptTemplate, BudgetEntry, RedactionRule . . . 32

Evaluation, search, and control entities . . . 34

Enums, indexes, and migration policy . . . 36

5 · API Specification 39

Conventions, auth, and the error envelope . . . 39

Endpoint catalog . . . 40
Server-sent events: the live update contract . . . 43

6 · AI Engine and Prompt Specification 44

Task registry . . . 44
Per-task specification and output schemas . . . 46
Routing policy and the retry ladder . . . 47
Budget ledger mechanics . . . 48
Redaction pipeline . . . 48
Prompt versioning and the evaluation harness . . . 49

7 · UX and Screen Specification 49

Design tokens . . . 50
Screen inventory . . . 50
Detailed layouts for the five complex screens . . . 52
Interaction rules and keyboard layer . . . 52

8 · Security, Privacy and Data Management 53

Threat model . . . 53
Data classification and the egress contract . . . 55
Key management . . . 55
Backup, restore, and retention . . . 55

9 · Testing and Quality Plan 56

Test pyramid and tooling . . . 57
Fixtures: the golden set . . . 57
Coverage matrix and quality gates . . . 58

10 · Operations and Deployment Runbook 60

Prerequisites and install . . . 60
First-run wizard and model provisioning . . . 61
Upgrade procedure . . . 61
Backup, restore, and monitoring . . . 61
Troubleshooting playbook . . . 61

11 · Configuration Reference 63

Environment variables 63
models.yaml: the model catalog 65
Setting keys and tuning knobs 65

12 · Build Orders - The Execution Plan 66

Phase 0 - Foundation 67
Phase 1 - Ingestion and transcription 67
Phase 2 - Analysis engine 68
Phase 3 - Deal Intelligence 69
Phase 4 - Coaching and Library 70
Phase 5 - Hardening and polish 71
Project definition of done and change control 72

1 · Dossier Guide and Completeness Audit

This volume is the second and final planning artifact for AMBER. The first, the Architecture and Product Blueprint (v1.0, 22 pages), decided what the system is: a Next.js 16 monolith over SQLite, a local-first hybrid AI engine with five provider adapters, four feature pillars, and a phased roadmap. This dossier answers a different question: can I sit down tomorrow and start building without inventing anything? For that, architecture is not enough. A build needs requirements with acceptance criteria, a column-level data dictionary, an API contract, prompt specifications with output schemas, a screen inventory, a threat model, a test plan, an operations runbook, a configuration reference, and work orders small enough to finish in one sitting. Version 1 contained none of those things, because it was a blueprint - a decision document, not a specification.

The brief for this volume was blunt: audit what is missing, close every gap, and produce every professional document the build needs so that no further planning is required. The twelve chapters that follow are those documents, consolidated into one dossier for daily use at the desk. Each chapter is self-contained and written to be read alone during the relevant phase of the build. Nothing in this volume reverses an architectural decision from v1; every gap is closed in the direction v1 already pointed. When the last chapter ends, the plan is complete, and the only remaining activity is execution.

How to use this dossier

Read the dossier in two passes. Before writing any code, read Chapters 1 through 3 once, end to end, to load the product shape into your head: what is being built, what each flow does, and what done means. Then switch to reference mode: build in the order Chapter 12 prescribes, and open the relevant specification chapter only when a work order tells you to. The document map below is the routing table - it names the traditional document each chapter replaces, and the moment in the build when you will need it. Do not try to hold the whole dossier in memory; that is what the work orders in Chapter 12 are for.

Ch.	Traditional document	You will open it when...
1	Completeness audit + risk register	You doubt the plan, or want to check nothing was forgotten (keep this table bookmarked)
2	Product requirements document (PRD)	Writing any feature: the acceptance criteria are the definition of done for every work order
3	Functional specification	Implementing any flow, state machine, or business rule (MEDDIC, risk flags, rubric, search)
4	Data dictionary / schema reference	Writing Prisma models, migrations, or any query that touches a column you cannot quite remember
5	API specification	Writing an API route or consuming one from the UI; the contract both sides code against
6	AI engine + prompt specification	Implementing any of the 12 AI tasks, the router, budget enforcement, or the eval harness
7	UX and screen specification	Building any of the 20 screens; layout, components, states, and shortcuts live here
8	Security and privacy plan	Handling keys, redaction, egress, backup, retention - read once before Phase 0, then consult
9	Testing and quality plan	Writing tests, fixtures, or the eval suite; coverage targets and thresholds live here
10	Operations and deployment runbook	Installing, upgrading, backing up, restoring, or troubleshooting the running app
11	Configuration reference	Wiring .env or models.yaml, or tuning any knob without re-reading code
12	Execution plan (work orders)	Every single working session: pick the next work order, build to its definition of done

Table 1.1 - Document map: what each chapter is, and when you will open it

The honest audit: what v1 did not contain

An honest answer to "are we sure this is the best we got" starts by admitting what was not there. The v1 blueprint was strong exactly where blueprints are supposed to be strong: the monolith decision, the provider-blind pipeline, the routing policy with enumerated escalation triggers, the cost model, and the model catalog were all specified to a level that survived review. But a blueprint deliberately stays above the implementation waterline, and sixteen things that a real build needs were left above that line with it. Each of them, left open, would have cost an unplanned decision in the middle of a coding session - which is exactly the kind of decision that gets made badly. The register below lists all sixteen, why each one matters, and the chapter of this dossier that closes it.

ID	Gap in v1	Why it matters	Severity	Closed in
G01	No PRD: features described, but no testable acceptance criteria	Without criteria you cannot distinguish done from almost-done; every work order inherits the ambiguity	Critical	Ch. 2
G02	Data model at entity level only (table names, no columns)	The first week of coding stalls on schema arguments you have with yourself at midnight	Critical	Ch. 4
G03	No API contract	UI and pipeline drift apart; integration happens by memory instead of by document	Critical	Ch. 5
G04	No prompt specs or output schemas for the AI tasks	The heart of the product - twelve AI tasks - had inputs and outputs but no interfaces	Critical	Ch. 6
G05	No evaluation harness for AI quality	No way to know whether a prompt or model change made extraction better or worse	Critical	Ch. 6 + 9
G06	Pipeline stages listed, but no state machine semantics	Resumability and crash-safety were claimed as guarantees but never specified as transitions	High	Ch. 3
G07	Design system without a screen inventory	Warm Calm tokens exist, but no screen list, layouts, or per-screen states; UI work would start with improvisation	High	Ch. 7
G08	No threat model or key-management detail	"Local-first" is a slogan until the threats are enumerated and each is answered in design	High	Ch. 8
G09	No test strategy or fixture set	Quality would depend on mood; regressions in the pipeline would be found by users (there is one: you)	High	Ch. 9
G10	No operations runbook	Install, first-run, upgrade, backup, restore, and troubleshooting were all implied, none written	High	Ch. 10
G16	Roadmap was phase-level, no work orders with definitions of done	"Start Phase 0" would immediately raise forty questions the plan had not answered	High	Ch. 12
G11	No configuration reference	Environment variables and the model catalog were implied by architecture but never enumerated	Medium	Ch. 11
G12	Budget cap described, ledger mechanics unspecified	Cost enforcement was a policy sentence, not a table, a transaction, and a hard stop	Medium	Ch. 6
G13	No schema migration or versioning policy	The schema will evolve monthly; without policy, migrations become archaeology	Medium	Ch. 4
G14	No retention or backup policy	Data management (what is kept, where, how copied, how restored) was left to chance	Medium	Ch. 8 + 10
G15	Redaction promised, rule catalog undefined	The privacy promise had no implementable pattern list, no entity types, no test pad	Medium	Ch. 6 + 8

Table 1.2 - Gap register: what the v1 blueprint lacked, and where each gap closes

The severity column is deliberately unforgiving: a gap is Critical when leaving it open would corrupt the build itself - wrong acceptance criteria, a schema you redo, an AI layer you cannot test - and High when it would reliably cost hours mid-phase. The absence of Medium-or-lower gaps below this line is itself a finding: v1 got the architecture right, and everything still missing was specification, not design. That is the healthiest possible position for a volume-two document to inherit.

The 360-degree completeness matrix

The brief demanded a full-360 plan - a demonstrable guarantee that nothing a professional build requires is left unwritten. The matrix below is that guarantee. It enumerates every document type a project of this scale conventionally produces, states what v1 covered, what this dossier covers, and the resulting status. The rule used to build it: if any row had ended Partial or Missing after this dossier, the dossier would not have shipped. Every row ships Full. Planning ends here.

Professional document	v1.0	This dossier	Where
Product vision and positioning	Full	Referenced	Ch. 2 opening
System architecture and rationale	Full	Unchanged	v1 Ch. 3-5
Technology selection with alternatives	Full	Unchanged	v1 Ch. 3
Cost model (local / hybrid / cloud vs Gong)	Full	Operationalized	Ch. 6 budget ledger
Product requirements with acceptance criteria	Missing	Full	Ch. 2 (FR-001..FR-024, NFR-01..NFR-14)
Functional flows and state machines	Partial	Full	Ch. 3
Data dictionary (column level)	Partial	Full	Ch. 4 (19 tables + enums + indexes)
API specification	Missing	Full	Ch. 5 (approx. 40 endpoints + SSE)
AI task and prompt specification	Partial	Full	Ch. 6 (T01-T12 with schemas)
AI evaluation and QA plan	Missing	Full	Ch. 6 + Ch. 9
Design system (tokens, components)	Partial	Full	Ch. 7
Screen inventory and layouts	Missing	Full	Ch. 7 (20 screens)
Security threat model	Missing	Full	Ch. 8
Privacy, retention, and consent position	Partial	Full	Ch. 8
Test strategy, fixtures, coverage matrix	Missing	Full	Ch. 9
Operations and deployment runbook	Missing	Full	Ch. 10
Configuration reference	Missing	Full	Ch. 11
Execution plan with definitions of done	Partial	Full	Ch. 12 (WO-001..WO-040)
Risk register with mitigations	Missing	Full	Ch. 1
Glossary of terms	Partial	Full	Ch. 2 closing

Table 1.3 - The 360-degree completeness matrix: every professional document, covered

Risk register: the things most likely to actually hurt

The most important things to worry about at this stage are not missing features - they are the failure modes of a solo, multi-month build. Ten risks account for nearly all realistic damage. Each is stated with its likelihood, its impact if ignored, the mitigation already built into this dossier, and where that mitigation lives. This register is the standing answer to "what are we missing": after the sixteen gaps above are closed, what remains risky is execution, not planning, and every execution risk below has an owner in a chapter.

ID	Risk	Likelihood	Impact	Mitigation	Lives in
R01	Transcription quality on real recordings (accents, codec noise, quiet voices) below expectation	Medium	High	Golden-set eval gate before prompt work; loudness normalization stage; whisper model-size ladder with measured RTF and accuracy	Ch. 6, 9
R02	Scope creep past the four confirmed pillars	High	High	MoSCoW freeze in Ch. 2; new ideas go to a v2 list, never into the current plan; work orders are immutable scope units	Ch. 2, 12
R03	Local LLMs break JSON discipline (invalid JSON, hallucinated fields)	High	Medium	Strict output schemas, one repair-retry, quality gates, then escalation to cloud per policy; all specified per task	Ch. 6
R04	Motivation decay on a solo multi-month build	Medium	High	Forty work orders sized 1-3 focus blocks; every order ships something visible; phase ends are demoable states	Ch. 12
R05	Hardware limits (8-16 GB RAM, no GPU) squeeze local models	Medium	Medium	Model-size ladder with 4-bit quantization defaults; CPU-first settings; cloud escalation per routing policy	Ch. 6, 10
R06	Cloud spend runs away silently	Low	Medium	Monthly budget cap enforced in the ledger transaction itself (hard stop), spend visible on every screen footer	Ch. 6
R07	SQLite corruption or accidental deletion destroys history	Low	High	Scheduled VACUUM INTO backups plus media-dir sync; restore procedure tested in the runbook dry-run	Ch. 8, 10
R08	Ollama model catalog drifts (names change, models deprecated)	Medium	Low	Catalog lives in models.yaml, not code; provider health screen surfaces breaks; swap is a config edit	Ch. 11
R09	Privacy accident: audio bytes reach a cloud provider	Low	Critical	Egress contract: STT is local-only at the code-path level; an integration test asserts zero audio bytes in every cloud adapter request log	Ch. 8, 9
R10	Upload friction quietly kills daily usage	Medium	High	Drag-drop everywhere plus optional watch-folder (Should tier); dogfooding checkpoint at end of Phase 2 measures real usage	Ch. 3, 12

Table 1.4 - Risk register: top ten risks, each mitigated inside this dossier

With the gap register closed and the risk register owned, the honest answer to the opening question is: yes - this is now the best plan this project can have without starting to build. The architecture was validated in

v1; the specification layer is now complete; the remaining uncertainty is concentrated exactly where uncertainty belongs, in execution. The eleven chapters that follow are the specification layer, in the order you will consume it.

2 · Product Requirements Document

This chapter is the single source of truth for what AMBER v1 must do and what "done" means for each piece of it. Every requirement is written as a testable statement, because the primary consumer of this document is not a stakeholder - it is the work orders in Chapter 12, each of which cites one or more requirement IDs as its definition of done. If a behavior is not in this chapter, it is not in v1. If it is in this chapter, it ships with acceptance criteria or it does not ship at all. That discipline is what allows a solo build to stay honest across months: there is no product manager to negotiate with, so the PRD negotiates with you.

Vision, persona, and jobs to be done

The vision statement from v1 stands unchanged: AMBER is a private, single-user revenue intelligence platform - a personal, better-than-Gong - where every architectural advantage comes from having exactly one user. The persona is a solo seller (a founder, independent rep, or consultant) who runs five to fifteen recorded conversations a week, works from a mid-range laptop with 8-16 GB of RAM, and cares about privacy both on principle and pragmatically: these recordings contain real customers saying real things. Their existing tools are a CRM they under-maintain, a notes app that decays, and memory, which fails. AMBER replaces the memory failure with infrastructure.

The jobs-to-be-done, in the persona's own words, anchor every requirement that follows. Each maps to a pillar, and no requirement exists that serves none of them.

- JTBD-1: "When a call ends, I want the details captured for me, so I do not lose what was said while it is still fresh." (Core Loop)
- JTBD-2: "When I am preparing a follow-up, I want a draft that already reflects the call, so I send it in two minutes instead of twenty." (Core Loop)
- JTBD-3: "When I look at my pipeline, I want to know which deals are stalling and why, so I spend attention where it pays." (Deal Intelligence)
- JTBD-4: "When I review how I sell, I want to see my patterns improving and my best moments saved, so I compound instead of repeat." (Coaching)
- JTBD-5: "When I remember something was said but not where, I want to find the exact moment, so I can act on it." (Library)

Scope: MoSCoW

The scope freeze below is the anti-creep instrument. The Must row is the v1 contract: it defines the shipped product. Should items are built only after every Must in their phase is done, and only if they cost what the estimate says. Could items are the acknowledged v2 backlog - writing them down is how they stay out of v1. Won't items are permanently out for v1 and are listed so that their absence is a decision, not an omission. When a new idea appears mid-build (it will), it enters the Could list or it does not happen; that single rule is R02's mitigation.

Tier	Items
Must	Core Loop end-to-end (upload, transcribe, analyze, notes, follow-up draft); MEDDIC extraction with evidence; risk flags; pipeline board with deal timelines and health; scorecard with six-dimension rubric; coaching trends; best-practice library; semantic search; local STT; hybrid routing with budget cap; redaction before cloud text; provider health; backup and restore
Should	Eval console UI; watch-folder ingestion; word-level timestamps; provider test buttons; spend chart; export bundle; cold-start importer (bulk upload of old recordings)
Could	Cross-call topic clustering; dark theme; mobile read-only reader; calendar-bot capture; CRM one-way export; multiplayer never
Won't	Multi-user anything (roles, sharing, leaderboards); CRM synchronization; forecasting; live/streaming capture; mobile authoring; telemetry or accounts of any kind; monetization

Table 2.1 - MoSCoW scope freeze for AMBER v1

Functional requirements - Core Loop

Functional requirements are grouped by pillar and numbered FR-001 through FR-024. Acceptance criteria are written to be checked mechanically where possible (numbers, states, statuses) and by inspection where not. Each work order in Chapter 12 ends by citing the FR IDs it satisfies; a phase is complete when all its FRs pass. The criteria assume the reference machine defined in Chapter 10 (8-core CPU, 16 GB RAM, no GPU) unless stated otherwise.

ID	Requirement	Acceptance criteria
FR-001	Upload accepts common recording formats with limits	mp3, mp4, m4a, wav, webm accepted up to 500 MB and 3 hours; anything else rejected with error code VALIDATION_ERROR and a readable message; no partial files persisted on rejection
FR-002	Duplicate detection via content hash	A file whose SHA-256 matches an existing asset links to the existing call and shows its results; no second transcription job is created; the UI states the duplicate explicitly
FR-003	Transcription completes within the time budget	A 30-minute reference recording reaches the TRANSCRIBED state within 15 minutes on the reference machine (RTF at or under 0.5), including normalization
FR-004	Speaker-attributed transcript with timestamps	Every segment has a speaker label, start and end in milliseconds, and text; labels are REP or PROSPECT after the relabel pass, with UNKNOWN permitted; segments cover at least 95% of spoken-word audio duration
FR-005	Talk metrics computed arithmetically	Talk ratio, question count, and longest monologue are computed from segment timings and text, never by model estimation; talk ratio matches a hand-computed check within 1 percentage point on the golden set
FR-006	Summary and structured notes per call	Every READY call has a 5-8 sentence summary and a notes draft (decisions, open questions, action items); notes are editable and the edit history is the user's, not the model's
FR-007	Follow-up email draft	Every READY call has a draft with subject and body, in the configured tone, referencing at least one concrete call detail; copyable in one click; status tracked DRAFT through SENT
FR-008	Pipeline is crash-safe and resumable	Killing the app during any stage and restarting loses at most the in-flight stage; the call re-enters at the last completed stage; no stage ever runs twice against the same input on resume

Table 2.2 - Core Loop requirements and acceptance criteria

Functional requirements - Deal Intelligence, Coaching, Library, Platform

The remaining pillars carry the differentiating intelligence. Deal Intelligence requirements define extraction honesty (unknown is a valid answer, silence is not) and the board mechanics. Coaching requirements define the rubric contract and the trend surface. Library requirements define search behavior and its latency budget. Platform requirements are the trust layer: routing transparency, budget enforcement, redaction, and the logging that makes every AI decision auditable after the fact.

ID	Requirement	Acceptance criteria
FR-009	MEDDIC extraction with evidence	All six fields extracted; each carries a value or an explicit NOT_FOUND, a confidence 0-1, and links to evidence segments; at least 5 of 6 fields correct on the golden set (field-level F1 at or above 0.85)
FR-010	Risk flags raised deterministically from signals	Flags: NO_NEXT_STEP, SINGLE_THREADED, COMPETITOR_MENTIONED, PRICING_FRICTION, TIMELINE_SLIP; each flag links to the segment that triggered it; no flag without evidence
FR-011	Pipeline board with drag-to-stage	Deals render in stage columns (LEAD through WON/LOST); drag updates stage persistently in one action; board loads under 1 second with 50 deals
FR-012	Deal timeline auto-built	Every call, confirmed insight, stage change, and note on a deal appears in one reverse-chronological timeline; entries link to their sources
FR-013	Deal health score	A 0-100 score computed from stage, MEDDIC completeness, next-step existence, and recency of activity; recomputed on every new analyzed call; formula is fixed in Ch. 3 and versioned

Table 2.3 - Deal Intelligence requirements

ID	Requirement	Acceptance criteria
FR-014	Six-dimension scorecard per call	Dimensions per the Ch. 3 rubric (rapport, discovery, value articulation, objection handling, next step secured, talk balance); each dimension scores 0-10 with an evidence link; overall is the weighted sum scaled to 100
FR-015	Trends over time	Scorecard trend charts by dimension and overall across a selectable window; empty state renders a calibration prompt, not a blank chart, until 3 calls exist
FR-016	Best-practice library from your own calls	User-promoted moments and model-suggested candidates live in one library; each entry carries source call and segment links; entries are searchable
FR-017	Comparison against own top quartile	Any scorecard can be compared against the user's top-quartile calls for the same dimension, with the metric deltas shown

Table 2.4 - Coaching requirements

ID	Requirement	Acceptance criteria
FR-018	Semantic search across everything	One search box searches transcripts, segments, notes, best practices, and call summaries; results are ranked by cosine similarity with snippets and the matched text highlighted
FR-019	Search latency budget	Search returns in under 2 seconds against 1,000 indexed calls on the reference machine (brute-force cosine is the specified v1 implementation)
FR-020	Filters on search	Results filterable by deal, contact, date range, and speaker; filters compose; counts update live
FR-021	Provider health and model catalog	Settings shows each provider (Ollama, OpenAI, Anthropic, Gemini, compatible) with live health, available models, and a one-click test; model catalog comes from models.yaml, never hard-coded
FR-022	Budget enforcement	A monthly cloud budget cap is enforced at the ledger transaction: when a provider run would exceed the cap, the run is refused (BUDGET_EXCEEDED) and the task degrades to local or marks the call FAILED_ANALYZE per policy; current spend is visible in Settings and in the footer
FR-023	Every AI run logged	Each run records task, provider, model, tokens, cost, latency, status, and fallback chain in ProviderRun; the call detail page shows the runs that produced its outputs
FR-024	Redaction before cloud text	When a task escalates to cloud, its text payload passes the enabled RedactionRule set; the redaction preview screen demonstrates each rule on sample text; an integration test proves the pipeline (Ch. 9)

Table 2.5 - Library and Platform requirements

Non-functional requirements

Non-functional requirements carry the qualities that make the product trustworthy rather than merely functional. Each is expressed with a threshold, because a quality without a number is a hope. These thresholds are referenced by the performance tests in Chapter 9 and by the acceptance of the phases in Chapter 12; a build that misses an NFR threshold is not done, even if every FR passes.

ID	Category	Requirement	Threshold
NFR-01	Performance	Dashboard and board interactive quickly	Under 1 second to interactive on reference machine
NFR-02	Performance	API latency excluding AI work	p95 under 150 ms for all non-pipeline endpoints
NFR-03	Performance	Transcription speed (RTF)	At or under 0.5 for whisper small/medium on reference machine
NFR-04	Reliability	Pipeline crash safety	Zero data loss on kill at any stage; at most one stage re-run (FR-008)
NFR-05	Privacy	Audio egress	Zero audio bytes to any network interface, by construction, verified by test
NFR-06	Cost	Cloud spend control	Monthly spend never exceeds cap (hard stop at the ledger)
NFR-07	Usability	Routine task depth	Any routine task reachable in 3 clicks or fewer from the dashboard
NFR-08	Maintainability	Provider adapters small and uniform	Each adapter at or under 150 lines; new provider in 1 day or less
NFR-09	Portability	Runs on common dev machines	macOS arm64 and Linux x64, Node 22, no GPU required
NFR-10	Data safety	Backup and restore simple and tested	Backup in at most 2 commands; restore rehearsed in the runbook dry-run
NFR-11	Observability	AI decisions auditable	100% of AI runs logged with cost, latency, and fallback chain
NFR-12	Startup	App starts fast	Cold start to interactive under 10 seconds
NFR-13	Security	Key hygiene	API keys exist only in .env, never in the database, logs, or exports
NFR-14	Extensibility	Prompts are data, not code	All prompt templates editable in Settings; versioned with activation flag

Table 2.6 - Non-functional requirements with measurable thresholds

Out of scope, and the glossary

The out-of-scope list is short because Chapter 2 already froze it in the MoSCoW table: multi-user surfaces, CRM synchronization, forecasting, live capture, calendar bots, mobile authoring, and any form of telemetry or account system are not in v1. Their absence is a design position, restated here so that no future chapter quietly reintroduces them. The glossary below defines the working vocabulary for the rest of the dossier; terms are used consistently from here on, and where a chapter needs a more specific definition (for example, the exact rubric), it extends rather than redefines these.

Term	Definition
MEDDIC	Qualification framework: Metrics, Economic buyer, Decision criteria, Decision process, Identify pain, Champion. Ch. 3 defines the extraction contract per field
Talk ratio	Percentage of call duration spoken by the rep, computed arithmetically from segment timings, never estimated by a model
RTF	Real-time factor: processing time divided by audio duration; 0.5 means a 30-minute call processes in 15 minutes
Diarization	Attributing transcript segments to speakers; v1 uses pause-based clustering plus one LLM relabel pass
Golden set	The fixed evaluation corpus (transcripts with expected extractions) used by the eval harness to score prompt and model changes
Escalation	Routing a task from local to cloud when a policy trigger fires (context overflow, quality-gate failure, budget available and tier rule allows)
Quality gate	A mechanical check on AI output (schema validity, field coverage, confidence floor) that decides accept, repair, or escalate
Redaction	Applying the enabled RedactionRule set to text before it can leave the machine for a cloud provider
Egress contract	The privacy invariant: audio never leaves; only text leaves, only when policy allows, only after redaction
Health score	A 0-100 deal-level composite defined in Ch. 3 (stage, MEDDIC completeness, next step, recency)
Work order	The atomic unit of Chapter 12: a scoped task with key files, dependencies, a definition of done, and an estimate in focus blocks
Focus block	A 90-minute uninterrupted working session; all estimates in this dossier are in focus blocks
Adapter	A provider client conforming to the uniform interface in Ch. 6 (Ollama, OpenAI, Anthropic, Gemini, compatible)
Content-addressed	Media identified by the SHA-256 of its bytes; identity is intrinsic, dedup is free (FR-002)

Table 2.7 - Glossary of working terms

3 · Functional Specification - The Four Pillars

This chapter specifies behavior: the exact flows, states, and business rules that the four pillars implement. Together with the acceptance criteria in Chapter 2 it defines what the system does; Chapters 4 through 6 define how that behavior is stored, served, and computed. The chapter is deliberately concrete - every rule that an implementation could get wrong is written down here, including the failure paths. When code and this chapter disagree during a build, the resolution order is: fix the code, or file a change request against this chapter; never improvise a third behavior.

The Core Loop, step by step

The Core Loop is the product's spine: from a raw recording to reviewed, acted-upon intelligence. Ten steps, each with a defined actor, result, and failure path. The UI surfaces these steps as the call's progress rail (Ch. 7), so the vocabulary here is also the user-facing vocabulary. Steps S1 through S4 are mechanical infrastructure; S5 through S9 are where the hybrid AI engine does its work; S10 is the human close of the loop.

Step	Actor	Action	Result	Failure path
S1	User	Drop or pick a recording; optionally set title, deal, contact, date	Call row created in UPLOADED with linked MediaAsset	Bad format or over 500 MB: rejected pre-create with VALIDATION_ERROR; nothing persisted
S2	System	SHA-256 the bytes; check for an existing asset	Duplicate links to the existing call; no job enqueued	Hash collision (never in practice): newest wins, logged
S3	Worker	Normalize: ffmpeg strip video, resample 16 kHz mono, loudness-normalize	WAV in the media vault; state NORMALIZING then TRANSCRIBING	ffmpeg error: state FAILED, errorCode NORMALIZE_FAILED, retry offered
S4	Worker	Transcribe locally (whisper), cluster speakers, relabel clusters via one LLM pass	Transcript + segments; state ANALYZING	STT error or empty transcript: FAILED, TRANSCRIBE_EMPTY or TRANSCRIBE_FAILED
S5	Engine	Compute talk metrics arithmetically	Talk ratio, question count, longest monologue persisted	None (pure arithmetic over segments)
S6	Engine	Run analysis tasks T03-T07 per Ch. 6 policy (summary, MEDDIC, risks, scorecard, notes, follow-up draft)	Insights, scorecard, notes, draft; state INDEXING	Per-task retry ladder; terminal failure of a Must task: FAILED, ANALYZE_FAILED, partials kept
S7	Worker	Embed segments, summary, notes, best-practice candidates	Vectors persisted; state READY	Embedding failure is non-blocking: READY with a warning chip; re-index available
S8	User	Review: read transcript, confirm/dismiss insights, edit notes	Insight statuses set; state REVIEWED	None; review is optional and can happen any time
S9	User	Send the follow-up: edit draft, copy, mark SENT	FollowUpDraft status SENT with timestamp	None; sending happens in your mail client by design (no mail sender in v1)
S10	User	Archive the call when fully processed	State ARCHIVED; hidden from inbox, searchable forever	None; reversible

Table 3.1 - The Core Loop: steps, results, and failure paths

The call state machine

Every call moves through the state machine below, and the machine is the contract that makes FR-008 (crash safety) enforceable rather than aspirational. Two invariants carry all the weight. Persist-before-advance: a stage writes its complete output and advances the status column in one transaction, so a crash between stages leaves a clean, resumable boundary. Idempotent re-entry: every stage validates its input and safely no-ops if its output already exists, which is what makes retry-after-crash harmless. FAILED is terminal but retryable - retry re-enters at the failed stage with retryCount incremented, and three consecutive failures park the call with a persistent error card instead of looping. CANCELED is available from any non-terminal state and keeps all artifacts.

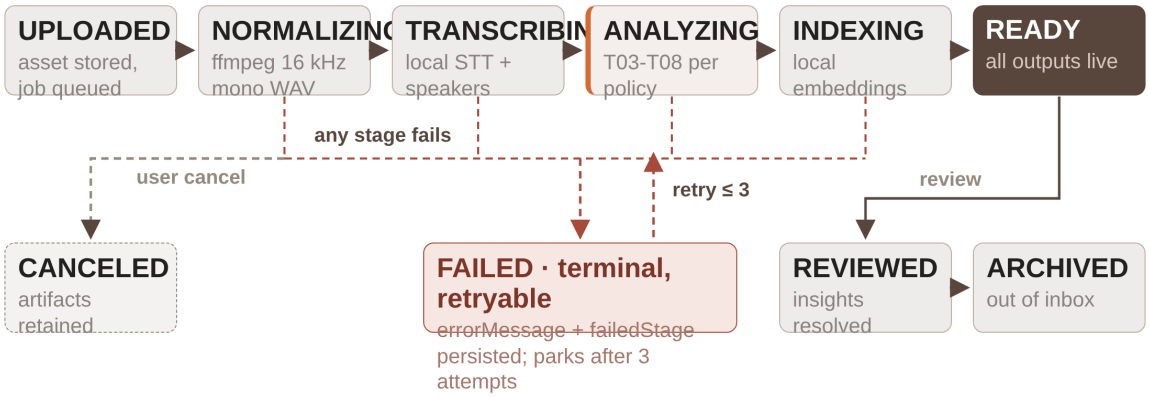


Figure 3.1 - The call processing state machine. Top lane: the happy path through the six pipeline stages. Red dashed edges are failure semantics (any stage can fail; retry re-enters the failed stage, parking after three attempts). Gray dashed edges are user-driven transitions.

State	Entered by	Exits	Artifacts persisted on exit
UPLOADED	S1 create, or retry reset	NORMALIZING (job picked)	MediaAsset row, original file in vault
NORMALIZING	Worker takes normalize job	TRANSCRIBING or FAILED	Normalized WAV (content-addressed)
TRANSCRIBING	Normalizer done	ANALYZING or FAILED	Transcript, Segments, speaker labels
ANALYZING	Transcriber done	INDEXING or FAILED	Insights, Scorecard, Note, FollowUpDraft, metrics
INDEXING	Analyzer done	READY (warning chip if embeddings failed)	Embedding rows
READY	Indexer done	REVIEWED, ARCHIVED, or re-ANALYZING (reanalyze action)	All outputs; call visible on all surfaces
REVIEWED	User confirms review complete	ARCHIVED, or back to READY on un-review	Insight statuses (CONFIRMED/DISMISSED), edited notes
ARCHIVED	User archives	READY (un-archive)	None new; inbox filter only
FAILED	Any stage error	Previous in-flight stage (retry), or stays FAILED after 3 attempts	errorMessage, failedStage, retryCount; partial artifacts kept
CANCELED	User cancels a queued/running call	Terminal (artifacts retained, deletable)	None new

Table 3.2 - State machine: states, entry, exit, and persisted artifacts

Deal Intelligence rules

MEDDIC extraction is governed by an honesty contract: the extractor must answer all six fields, and "not found in this call" is a valid, first-class answer - hallucinating a champion because the field exists is the failure mode the contract exists to prevent. Each field carries a confidence between 0 and 1 and links to the evidence segments that justify it; a field with confidence below 0.5 renders in the UI as "weak signal" rather than as fact. Table 3.3 defines what each field means operationally; Table 3.4 defines the five deterministic risk flags and their triggers; the health score formula follows as versioned pseudo-code, so that a future change is a version bump, not a mystery.

Field	Operational question	Evidence expectations
M - Metrics	What measurable outcome did the prospect state?	Quantified statements by the prospect ("cut onboarding from 6 weeks to 2"); rep-asserted numbers score lower
E - Economic buyer	Who signs, and did we engage them?	Names/roles tied to budget or approval language; "my CFO will need to see this" is a lead, not a confirmed EB
D - Decision criteria	What will they evaluate us against?	Explicit comparison criteria, evaluation plans, scoring language
D - Decision process	What is the path and timeline to signature?	Stated steps, committees, procurement stages, dates
I - Identify pain	What pain, in their words, did we surface?	Prospect-voiced pain with consequence language; pain repeated verbatim across calls strengthens confidence
C - Champion	Who is selling for us internally?	Promises of internal advocacy, volunteering to drive the process, forwarding behavior mentioned on the call

Table 3.3 - The MEDDIC extraction contract, field by field

Flag	Trigger	Severity	Surface
NO_NEXT_STEP	Latest analyzed call on the deal ends with no agreed next step or date in insights, notes, or draft	High	Deal card badge + deal detail banner
SINGLE_THREADED	All calls on the deal involve one contact, or one contact is the only engaged party for 21+ days	Medium	Deal card badge + contacts panel hint
COMPETITOR_MENTIONED	Prospect names an alternative (matched against a curated competitor list, configurable)	Medium	Deal timeline entry + insight card
PRICING_FRICTION	Objection language on price, budget, discounting, or procurement pushback detected in latest call	Medium	Insight card + coaching link
TIMELINE_SLIP	Expected close date passes without stage advance, or prospect states a delay	Medium	Deal card badge + board sort option

Table 3.4 - Risk flags: triggers, severity, and board surface

```
health_score v1 (recompute on every analyzed call):
  base      = stage_weight[stage]    # LEAD 20 QUALIFIED 35 PROPOSAL 55
                                     # NEGOTIATION 75 WON 100 LOST 0 PARKED 10
  meddic    = 25 * (filled_fields / 6)   # NOT_FOUND is not filled
  next_step = 15 if nextStep set and due within 14 days,
```

```
             else 5 if set, else 0
recency   = 15 if last activity within 7 days, 8 if within 14,
             3 if within 30, else 0
health    = clamp(round(0.45*base + meddic + next_step + recency), 0, 100)
```

Health score formula, version 1. Weights sum to 100; the formula is versioned in the Setting table so historical scores stay explainable.

Coaching specification

Every analyzed call receives a scorecard against a six-dimension rubric you own. The rubric is versioned; version 1 ships with the weights below. Four dimensions (discovery, value articulation, objection handling, next step) are scored by the LLM against the rubric text with evidence links; two (talk balance, and rapport signals) blend arithmetic metrics with LLM judgment so that the numbers that can be computed exactly are never guessed. The overall score is the weighted mean scaled to 100. Trends plot each dimension and the overall over time; until three calls exist, the coaching screen shows a calibration explanation instead of a spiky, meaningless chart (FR-015).

Dimension	Weight	Full marks (10) looks like	Inputs
Discovery	25%	Open questions surfaced pain, metrics, and process; prospect spoke unprompted about their world; at least 4 diagnostic questions	LLM-judged + question count
Value articulation	20%	Value stated in the prospect's metrics, tied to named pain; no unexplained jargon monologues	LLM-judged
Objection handling	15%	Objections acknowledged, isolated, and answered or parked honestly; no steamrolling	LLM-judged + objection insights
Next step secured	15%	A specific next step with a date agreed before the call ends	LLM-judged + NO_NEXT_STEP flag
Talk balance	15%	Rep talk ratio 40-60%; no monologue over 120 seconds; questions outnumber statements in discovery	Arithmetic from segments
Rapport	10%	Natural openings, names used, humor or warmth present without time theft	LLM-judged

Table 3.5 - Scorecard rubric v1: dimensions, weights, and what full marks look like

The best-practice library accumulates in two ways. Model-suggested candidates are moments the analysis pass marks as strong (a handled objection, a crisp discovery sequence) with links to their segments. User promotions are one-click saves from any transcript selection. Both live as BestPractice entries with source evidence, searchable and taggable (FR-016). Comparison against your own top quartile (FR-017) compares a selected call's dimension scores with the mean of your top-quartile calls - self-benchmarking replaces the team leaderboards an enterprise tool would offer.

Library specification

The Library indexes five resource types: transcript segments (the atomic unit), call summaries, notes, best practices, and confirmed insights. Segments are additionally indexed as rolling 3-segment windows so that a match can surface conversational context even when the exact words fall across a segment boundary. Embeddings are produced by the local model (T10, nomic-embed-text) and stored in the Embedding table; search is brute-force cosine over the vector column, filtered first by any active deal/contact/date/speaker filters, then ranked. That is deliberately un exotic: at 1,000 calls (roughly two years of heavy usage) the index holds on the order of 60,000 vectors, and brute-force cosine returns well inside the 2-second budget (FR-019). A vector index extension remains a v2 option, adopted only if measured latency says so.

- Chunking: one embedding per segment + one per rolling 3-segment window + one per summary/note/practice/insight; all deduplicated by (resourceType, resourceId, chunkIdx).
- Ranking: cosine similarity, then recency tiebreak; snippet is the matched chunk with query terms highlighted; a grouped view collapses multiple hits from the same call.
- Re-index triggers: new analysis (S7), note edit, best-practice create; deletion removes vectors in the same transaction as the source row.
- Search box behavior: plain text is embedded as a query vector; quoted phrases additionally run a substring pass and merge results; empty query shows recent calls.

Edge cases and the error matrix

The edge cases below are the ones a real corpus will produce within weeks. Each row fixes the behavior now so that no coding session has to decide it under pressure. The general principles, in order: never lose user data, never silently pretend success, always leave a retry path, and degrade gracefully when a stage's output is optional.

Scenario	System behavior	User sees
Mostly-silent audio (hold music, no-show)	Transcript may be empty or near-empty; if under 25 words, mark TRANSCRIBE_EMPTY instead of ANALYZING	Call card: "Nothing to transcribe" with audio retained
Very long recording (2-3 hours)	Process in full; progress rail shows stage elapsed time; RTF budget scales linearly	Normal states, longer wait, no timeout
Non-English call	STT detects language; analysis prompts include the language hint; UI states detected language	Normal outputs; language chip on the call
Corrupt or unreadable file	ffmpeg or STT fails; state FAILED with NORMALIZE_FAILED/TRANSCRIBE_FAILED; retry offered once automatically	Error card with stage and retry button
App killed mid-transcription	Job lock released on restart; stage re-enters; idempotency prevents duplicate artifacts	Nothing lost; call continues from stage
Ollama not running	Local LLM tasks fail fast; policy escalates text tasks to cloud if budget allows, else call parks in FAILED with PROVIDER_UNAVAILABLE	Warning banner + provider health link
Cloud key invalid or revoked	Adapter marks provider unhealthy for 15 minutes; routing skips it; visible in provider health	Provider row turns red in Settings
Budget exhausted mid-call	Remaining cloud tasks refused at the ledger (BUDGET_EXCEEDED); policy degrades to local; if local cannot serve, call parks	Spend meter red; call card names the refused task
Duplicate upload after processing	SHA-256 match links to the existing call including all results	"Already in your library" with link
Call linked to a deal that is later deleted	Deal delete is soft (status LOST/PARKED or archived flag); calls keep their timeline; hard delete warns and unlinks	Warning modal; data preserved
Same contact name across deals	Contacts are deal-scoped by design; no global merge in v1; search shows both	Both contacts listed with deal context
File over 500 MB or 3 hours	Rejected pre-upload with size rule stated	Inline rejection message
Reanalyze after prompt change	New insights created; prior ones superseded (status SUPERSEDED), not deleted - history preserved	"Re-analyzed" chip with diff view of insights
Embedding model switched	Re-index job enqueued for all READY calls; old vectors deleted on completion; progress visible	Settings shows re-index progress

Table 3.6 - Edge cases and the error matrix: what happens, and what you see

4 · Data Dictionary and Schema Reference

This chapter is the column-level contract for the SQLite database, written for the Prisma schema that Phase 0 creates and every later phase extends. Conventions used throughout: identifiers are cuid strings; timestamps are UTC DateTime and every table carries createdAt; money is integer cents; enums are Prisma enums backed by SQLite text with the values in Table 4.21; Json columns hold structured payloads whose shapes are specified in the Notes; and no column exists "for later" - every column is written by a specified code path in this dossier. Foreign keys use cascade rules chosen for data safety: deleting a parent that owns artifacts requires either soft-delete or an explicit unlink, so a stray delete can never silently destroy transcripts.

Core entities: Deal, Contact, Call, MediaAsset

The four core entities carry the deal graph and the ingestion spine. Deal and Contact are small and stable; Call is the stateful heart that every other table hangs from; MediaAsset is deliberately separate from Call so that content-addressed dedup (FR-002) is a relation, not a copy. Note the denormalized dealId on Insight - it exists because the board queries insights per deal and the join through Call would be run thousands of times per board render.

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
title	String	required	Human label, e.g. "Acme - Expansion Q4"
account	String	required	Company name; board groups by it in v2, free text in v1
stage	DealStage	default LEAD	Enum (Table 4.21); board column source
valueCents	Int?		Deal value in cents; null = unquantified
currency	String	default "EUR"	ISO 4217; display-only in v1
expectedCloseAt	DateTime?		Drives TIMELINE_SLIP and board sorting
healthScore	Int?		0-100 per Ch. 3 formula; null until first analyzed call
nextStep	String?		Plain text; NO_NEXT_STEP reads this plus call insights
nextStepDueAt	DateTime?		Next-step date; feeds health score
archivedAt	DateTime?		Soft delete; board hides archived deals
createdAt / updatedAt / lastActivityAt	DateTime	required	lastActivityAt bumped by calls, insights, stage changes

Table 4.1 - Deal

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
dealId	String	FK -> Deal, cascade	Contacts are deal-scoped in v1 (no global directory)
name	String	required	As used on calls
title	String?		Role, e.g. "Head of Ops"
email / phone	String?		Optional; never sent to cloud un-redacted (email is a redaction entity)
isChampion	Boolean	default false	Set by MEDDIC C extraction or manually
isEconomicBuyer	Boolean	default false	Set by MEDDIC E extraction or manually
notes	String?		Free text
createdAt / updatedAt	DateTime	required	

Table 4.2 - Contact

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
title	String	required	Default: file name without extension; editable
dealId	String?	FK -> Deal, set null	Unlinked calls are allowed (inbox-first workflow)
contactId	String?	FK -> Contact, set null	Primary counterpart on the call, if known
occurredAt	DateTime	required	Defaults to upload time; user-correctable
durationSec	Int?		Measured by ffprobe after normalize
source	CallSource	default UPLOAD	UPLOAD or IMPORT (bulk import, Should tier)
status	CallStatus	default UPLOADED	The state machine column (Table 3.2)
audioSha256	String?	index	Set at S2; the dedup key alongside MediaAsset
mediaAssetId	String?	FK -> MediaAsset, set null	Points at original bytes; normalized WAV is a second asset
language	String?		BCP-47 from STT detection, e.g. "en", "de"
errorMessage	String?		Human-readable terminal error for FAILED
failedStage	String?		Which stage failed, for retry re-entry
retryCount	Int	default 0	Consecutive failures; parks at 3
createdAt / updatedAt	DateTime	required	

Table 4.3 - Call

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
sha256	String	@unique	Content address of the exact bytes
filePath	String	required	Relative to MEDIA_DIR: ab/cd/. (two-level fan-out)
mimeType	String	required	As accepted at upload
sizeBytes	BigInt	required	Original size; drives the 500 MB rule
durationSec	Float?		From ffprobe when known
sampleRateHz / channels	Int?		Post-normalize audio facts
createdAt	DateTime	required	Assets are immutable; no updatedAt

Table 4.4 - MediaAsset

Transcript entities: Transcript, Segment

The transcript pair stores exactly what the STT stage produced, before any model touched it. Transcript is the whole-call record with provenance (which STT model and run produced it); Segment is the timestamped, speaker-attributed unit that every evidence link in the system points to. Segment ordering is enforced by a unique (transcriptId, idx) pair, and timings are integer milliseconds because float seconds produce off-by-one artifacts at boundaries.

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String	FK -> Call, @unique, cascade	One transcript per call; re-analysis supersedes via a new row and a status flag on Call
fullText	String	required	Plain text, segments joined by newlines
wordCount	Int	required	Computed at write time
language	String	required	Detected language
confidenceAvg	Float?		Mean STT confidence when the engine reports it
sttModel	String	required	e.g. "whisper-small" - provenance
providerRunId	String?	FK -> ProviderRun	The run that produced it
createdAt	DateTime	required	Immutable once written

Table 4.5 - Transcript

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
transcriptId	String	FK -> Transcript, cascade	
idx	Int	required	0-based order; @@unique([transcriptId, idx])
speaker	SpeakerLabel	required	REP, PROSPECT, SPEAKER_00/01 (pre-relabel), UNKNOWN
startMs / endMs	Int	required	Integer milliseconds; monotonic within a transcript
text	String	required	Segment text

Table 4.6 - Segment

Intelligence entities: Insight, Scorecard, Note, FollowUpDraft, BestPractice

These five tables hold everything the analysis stage (S6) produces and everything the user adds on top. Insight is deliberately generic over type: MEDDIC fields, risk flags, objections, buying signals, competitor mentions, and next steps are all Insights with different types, which lets the review queue, the evidence links, and the supersede-on-reanalyze logic be one mechanism instead of six. Scorecard is one row per call with the rubric version stamped on it, so a rubric change never rewrites history.

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String	FK -> Call, cascade	The call that produced it
dealId	String?	FK -> Deal, set null	Denormalized for board queries
type	InsightType	required	MEDDIC_M, MEDDIC_E, MEDDIC_D_CRITERIA, MEDDIC_D_PROCESS, MEDDIC_I, MEDDIC_C, RISK_FLAG, OBJECTION, BUYING_SIGNAL, COMPETITOR, PRICING_SIGNAL, TIMELINE_SIGNAL, NEXT_STEP, TOPIC
title	String	required	Short label, e.g. "Champion: Sarah (strong)"
content	String	required	The substance, 1-3 sentences
confidence	Float	required	0-1; below 0.5 renders as weak signal
evidenceSegmentIds	Json	required	Array of Segment ids; never empty for evidence-bearing types
status	InsightStatus	default ACTIVE	ACTIVE, CONFIRMED, DISMISSED, SUPERSEDED
supersedesId	String?	FK -> Insight	Set when reanalysis replaces this row
createdAt / updatedAt	DateTime	required	

Table 4.7 - Insight

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String	FK -> Call, @unique, cascade	One scorecard per call
rubricVersion	String	required	e.g. "v1" - matches a rubric definition in Setting
overall	Int	required	0-100, weighted mean per Ch. 3
dimensions	Json	required	[{key, score 0-10, weight, evidenceSegmentIds, comment} x 6]
talkRatioRep	Float	required	Percent, arithmetic (S5)
questionCount	Int	required	Arithmetic
longestMonologueSec	Int	required	Arithmetic
providerRunId	String?	FK -> ProviderRun	Provenance
createdAt	DateTime	required	Immutable; reanalyze writes a new row versioned by rubric

Table 4.8 - Scorecard

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String?	FK -> Call, cascade	Call-scoped or deal-scoped, exactly one set
dealId	String?	FK -> Deal, cascade	
body	String	required	Markdown-lite (bold, lists, links)
pinned	Boolean	default false	Pinned notes float to top of timelines
origin	String	default "user"	user or ai; AI notes are drafts until edited or accepted
createdAt / updatedAt	DateTime	required	

Table 4.9 - Note

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String	FK -> Call, @unique, cascade	One active draft per call; SENT history kept
channel	Channel	default EMAIL	EMAIL, LINKEDIN, OTHER - display hint in v1
subject	String	required	Drafted by T07, user-editable
body	String	required	Markdown-lite; references concrete call detail (FR-007)
tone	String	default "warm-professional"	Prompt knob for T07
status	FollowUpStatus	default DRAFT	DRAFT, EDITED, COPIED, SENT, DISCARDED
sentAt	DateTime?		User-marked; no mail sending in v1
createdAt / updatedAt	DateTime	required	

Table 4.10 - FollowUpDraft

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
title	String	required	e.g. "Price objection - park and reframe"
category	String	required	Discovery, Objections, Value, Next steps, Rapport, Other
body	String	required	The practice, written to be reusable
sourceCallId	String?	FK -> Call, set null	Where it came from, if promoted from a moment
evidenceSegmentIds	Json	default []	The moments that exemplify it
origin	PracticeOrigin	required	USER_PROMOTED or MODEL_SUGGESTED
status	PracticeStatus	default CANDIDATE	CANDIDATE, ACCEPTED, ARCHIVED - suggestions need your accept
tags	Json	default []	Free tags, searchable
createdAt / updatedAt	DateTime	required	

Table 4.11 - BestPractice

AI engine entities: ProviderRun, PromptTemplate, BudgetEntry, RedactionRule

The four engine tables make the hybrid AI layer auditable and controllable. ProviderRun is the flight recorder: one row per attempt of every AI task, including the fallback chain, so any output in the app can answer "which model, at what cost, produced this, and what did it replace". PromptTemplate turns prompts into versioned data (NFR-14); BudgetEntry is the ledger that enforces the monthly cap transactionally; RedactionRule is the editable catalog that the egress contract depends on.

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
taskId	String	required	T01-T12 per Ch. 6 registry
provider	Provider	required	OLLAMA, OPENAI, ANTHROPIC, GEMINI, COMPATIBLE, LOCAL_STT
model	String	required	Actual model id, e.g. "qwen3:8b"
promptTemplateId	String?	FK -> PromptTemplate	Template version used
callId	String?	FK -> Call, set null	Null for standalone tasks (e.g. library queries)
status	RunStatus	default PENDING	PENDING, RUNNING, OK, RETRIED, ESCALATED, FAILED
attempt	Int	default 1	1 = first try; 2+ = repair-retry
fallbackFromId	String?	FK -> ProviderRun	The failed local run this escalation replaced
inputTokens / outputTokens	Int?		Reported where available; local runs estimate
costUsd	Float	default 0	0 for local; cloud cost per price table in Setting
latencyMs	Int?		Wall-clock for the attempt
errorCode / error	String? / Text?		Machine code + human detail on failure
createdAt / finishedAt	DateTime / DateTime?	required / nullable	

Table 4.12 - ProviderRun

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
taskId	String	required	T03-T12 (LLM tasks only)
version	Int	required	@@unique([taskId, version])
systemPrompt	Text	required	Role and rules text
userTemplate	Text	required	Mustache-style placeholders: {{transcript}}, {{dealContext}}, {{language}}
outputSchema	Json	required	JSON Schema the response must satisfy (quality gate)
modelHint	String?		Preferred model class: small, medium, large
minContextChars	Int	default 0	Below this, task escalates by policy
active	Boolean	default false	Exactly one active version per task, enforced in service layer
createdAt	DateTime	required	

Table 4.13 - PromptTemplate

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
month	String	required, "YYYY-MM"	The ledger partition
providerRunId	String	FK -> ProviderRun, @unique	One entry per billable run; local runs write nothing
amountUsd	Float	required	Positive; written in the same transaction as the run's OK status
runningTotalUsd	Float	required	Denormalized after-write total for the month; cap check reads this
note	String?		e.g. "gpt-4o-mini summary for call xyz"
createdAt	DateTime	required	

Table 4.14 - BudgetEntry

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
name	String	required	Human label, e.g. "Email addresses"
patternType	PatternType	required	REGEX or NER (NER = local model tags entities)
pattern	String?		Required for REGEX; e.g. email, E.164 phone, IBAN, card patterns
entityType	EntityType?		Required for NER: EMAIL, PHONE, CREDIT_CARD, IBAN, URL, PERSON, ORG, AMOUNT, CUSTOM
action	RuleAction	default MASK	MASK ([EMAIL]), REPLACE (fixed token), SKIP (strip sentence)
replacement	String?		Used by REPLACE, e.g. ""
scope	RuleScope	default CLOUD_ONLY	CLOUD_ONLY or ALL (ALL also affects exports)
priority	Int	default 100	Lower runs first; ordering matters for overlapping rules
enabled	Boolean	default true	
createdAt / updatedAt	DateTime	required	

Table 4.15 - RedactionRule

Evaluation, search, and control entities

The remaining tables close the loop on quality, search, and settings. EvalCase and EvalRun persist the golden-set harness so a prompt experiment is a first-class, repeatable operation rather than a vibe. Embedding stores vectors as BLOBs with dimension stamps so a model switch can find and replace its own rows. Setting is the typed key-value table for everything the UI can change at runtime - budget cap, rubric definitions, competitor list, price tables - so that configuration that must be auditable lives in the database, while infrastructure configuration stays in the environment (Ch. 11 draws that line explicitly).

Table	Column	Type / constraints	Notes
EvalCase	id / taskId / name	cuid / String / String	A golden case: task, human label
EvalCase	fixturePath	String, required	Path under fixtures/eval: transcript input JSON
EvalCase	expectedJson	Json, required	Expected output structure per task metrics (Ch. 9)
EvalCase	notes	String?	Why this case exists (e.g. "heavy accent, pricing objection")
EvalRun	id / caseId	cuid / FK -> EvalCase	One scored execution
EvalRun	provider / model	String / String	What was tested
EvalRun	scores	Json	Per-metric scores from the harness (Ch. 9 defines metrics)
EvalRun	passed / createdAt	Boolean / DateTime	Against the task threshold

Table 4.16 - EvalCase and EvalRun

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
resourceType	EmbeddablеType	required	SEGMENT, SEGMENT_WINDOW, CALL_SUMMARY, NOTE, BEST_PRACTICE, INSIGHT
resourceId	String	required	Poly reference; @@index([resourceType, resourceId])
chunkIdx	Int	default 0	For multi-chunk resources
text	String	required	The embedded text (rebuildable source)
model	String	required	e.g. "nomic-embed-text"; model switch keys off this
dim	Int	required	Vector length, stamped per row
vector	Bytes	required	Float32 array serialized; brute-force cosine in service layer
createdAt	DateTime	required	

Table 4.17 - Embedding

Column	Type	Constraints	Notes
key	String	@id	Dot-namespaced: budget.monthlyCapUsd, rubric.v1, competitors.list, prices.openai, ui.tone
value	Json	required	Shape is defined per key in Ch. 11 defaults table
updatedAt	DateTime	required	

Table 4.18 - Setting

Column	Type	Constraints	Notes
id	String	@id, cuid	Primary key
callId	String?	FK -> Call, cascade	Null for non-call jobs (re-index, eval)
type	JobType	required	NORMALIZE, TRANSCRIBE, ANALYZE, INDEX, REINDEX, EVAL
status	JobStatus	default QUEUED	QUEUED, RUNNING, DONE, FAILED, CANCELED
priority	Int	default 5	User-triggered retries run before bulk imports
attempts / maxAttempts	Int	default 0 / 3	Exponential backoff between attempts
runAfter	DateTime?		Backoff and scheduled re-index
lockedAt	DateTime?		Worker lease; stale locks reclaimed on boot (crash recovery)
lastError	Text?		
createdAt / updatedAt	DateTime	required	

Table 4.19 - Job

Enums, indexes, and migration policy

Every enum in the schema is listed below with its exact values; the values are part of the API surface and are referenced by name throughout this dossier. The index set is intentionally small - SQLite serves single-user read patterns well, and the indexes below are exactly the ones the specified queries need: the board, the inbox, the review queue, the ledger, and the search filters. Anything not listed is a full scan and is fine at this scale.

Table	Index	Serves
MediaAsset	sha256 @unique	Dedup lookup at S2
Call	status, createdAt	Inbox: active calls newest-first
Call	dealId, occurredAt	Deal timeline assembly
Call	audioSha256	Duplicate check fast path
Transcript	callId @unique	One-per-call fetch
Segment	transcriptId, idx @unique	Ordered transcript read
Insight	dealId, type, status	Board risk badges and review queue
Insight	callId, status	Call detail insights panel
ProviderRun	callId, createdAt	Provenance panel on call detail
ProviderRun	createdAt	Spend reports and health history
BudgetEntry	month, createdAt	Ledger cap check and spend chart
Embedding	resourceType, resourceId	Delete-on-source-delete; re-index by model
Job	status, priority, runAfter	Worker queue poll (order: priority, then runAfter)
PromptTemplate	taskId, active	Active-template lookup per task

Table 4.20 - Index strategy: every index, and the query it serves

Enum	Values
DealStage	LEAD, QUALIFIED, PROPOSAL, NEGOTIATION, WON, LOST, PARKED
CallSource	UPLOAD, IMPORT
CallStatus	UPLOADED, NORMALIZING, TRANSCRIBING, ANALYZING, INDEXING, READY, REVIEWED, ARCHIVED, FAILED, CANCELED
SpeakerLabel	SPEAKER_00, SPEAKER_01, REP, PROSPECT, UNKNOWN
InsightType	MEDDIC_M, MEDDIC_E, MEDDIC_D_CRITERIA, MEDDIC_D_PROCESS, MEDDIC_I, MEDDIC_C, RISK_FLAG, OBJECTION, BUYING_SIGNAL, COMPETITOR, PRICING_SIGNAL, TIMELINE_SIGNAL, NEXT_STEP, TOPIC
InsightStatus	ACTIVE, CONFIRMED, DISMISSED, SUPERSEDED
Channel	EMAIL, LINKEDIN, OTHER
FollowUpStatus	DRAFT, EDITED, COPIED, SENT, DISCARDED
PracticeOrigin	USER_PROMOTED, MODEL_SUGGESTED
PracticeStatus	CANDIDATE, ACCEPTED, ARCHIVED
Provider	OLLAMA, OPENAI, ANTHROPIC, GEMINI, COMPATIBLE, LOCAL_STT
RunStatus	PENDING, RUNNING, OK, RETRIED, ESCALATED, FAILED
PatternType	REGEX, NER
EntityType	EMAIL, PHONE, CREDIT_CARD, IBAN, URL, PERSON, ORG, AMOUNT, CUSTOM
RuleAction	MASK, REPLACE, SKIP
RuleScope	CLOUD_ONLY, ALL
EmbeddableType	SEGMENT, SEGMENT_WINDOW, CALL_SUMMARY, NOTE, BEST_PRACTICE, INSIGHT
JobType	NORMALIZE, TRANSCRIBE, ANALYZE, INDEX, REINDEX, EVAL
JobStatus	QUEUED, RUNNING, DONE, FAILED, CANCELED

Table 4.21 - Enum catalog (values are API-stable names)

Migration policy is four rules, chosen so that schema evolution stays boring. One: Prisma migrate is the only path; hand-edited SQL never touches dev.db, and migrations are committed files. Two: migrations are additive-first - new columns are nullable or defaulted, destructive changes ship as a two-step (add, backfill, then remove in the next release) so a botched upgrade never loses data. Three: every migration that changes analysis-affecting structure bumps a schemaFeature flag in Setting, and the eval harness runs against the golden set after any such migration before the app is considered upgraded. Four: the seed script (fixtures + default settings + starter prompt templates) is idempotent and re-runnable, so a fresh

clone reaches a working state with one command.

5 · API Specification

The API is the contract between the UI and the server inside one Next.js process, and writing it down is what keeps both sides honest across forty work orders. Conventions first, then the full catalog by resource. All endpoints are under /api, return JSON except where stated, and use standard HTTP status codes plus the AMBER error envelope below. The API is single-user: authentication is a session cookie issued by the local sign-in screen (a passphrase from .env, not a user database) - enough to keep a browser on the same machine honest, which is the threat model in Chapter 8.

Conventions, auth, and the error envelope

Requests are JSON unless the endpoint is multipart (upload) or SSE (events). Responses wrap collections in { items, total } for pagination consistency, with cursor-free limit/offset (personal scale never needs cursors). Dates are ISO 8601 UTC. IDs are the cuid strings from Chapter 4. Errors always use the envelope in Table 5.2 - a code from the fixed catalog, a human message, and optional details - so the UI can branch on error.code without parsing prose. Every mutating endpoint validates with a zod schema shared between client and server; the shared schemas are the real API contract and live in one directory.

Aspect	Rule
Base path	/api (Next.js App Router route handlers)
Auth	Cookie session from POST /api/auth/login (passphrase from .env); all endpoints except /health require it
Content type	application/json; multipart/form-data for upload; text/event-stream for SSE
Collections	{ items: T[], total: number } with ?limit (default 50, max 200) and ?offset
Mutations	zod-validated; 400 with VALIDATION_ERROR and field details on failure
Idempotency	Upload is naturally idempotent via SHA-256 (FR-002); retries are safe
Time	ISO 8601 UTC everywhere; the UI renders in local time

Table 5.1 - Endpoint conventions

Code	HTTP	Meaning
VALIDATION_ERROR	400	Request failed schema validation; details carry field errors
NOT_FOUND	404	Resource does not exist (or is archived where it matters)
UNAUTHORIZED	401	No or expired session
PIPELINE_CONFLICT	409	Action invalid for the call's current state (e.g. reanalyze on FAILED without retry); details carry current status
PROVIDER_UNAVAILABLE	503	All providers for the task are down or unhealthy
BUDGET_EXCEEDED	402	Cloud run refused by the ledger; details carry month spend and cap
UNSUPPORTED_MEDIA	415	Format/size/duration rule failed at upload (FR-001)
RATE_LIMITED_LOCAL	429	Client-side throttle (only on provider test and eval run endpoints)

Table 5.2 - Error envelope and the error code catalog

Endpoint catalog

The catalog is grouped by resource and uses one compact notation: METHOD /path - purpose, followed by the request and response contract where it is not obvious from the table. Where an endpoint exists to serve a screen from Chapter 7, the screen is named, so UI work orders can code against this chapter alone. About forty endpoints cover the entire v1 surface; if a work order seems to need a new one, it cites the requirement that justifies it or it does not ship.

Method / Path	Purpose	Contract notes
POST /api/calls	Upload a recording (S1)	multipart: file + optional {title, dealId, contactId, occurredAt}; 202 {call, duplicate: bool}; 415 UNSUPPORTED_MEDIA on rule failure
GET /api/calls	Inbox list	filters: status (multi), dealId, q (title match), from, to; sorted occurredAt desc
GET /api/calls/:id	Call detail header	Includes status, progress stage, errorMessage, language, metrics
PATCH /api/calls/:id	Edit metadata	title, dealId, contactId, occurredAt; 409 if CANCELED
DELETE /api/calls/:id	Delete call + artifacts	Confirms; removes media assets only when no other call references them (dedup-safe)
POST /api/calls/:id/retry	Retry from failed stage	409 PIPELINE_CONFLICT unless FAILED; resets retryCount
POST /api/calls/:id/reanalyze	Re-run analysis (S6)	409 unless READY/REVIEWED; supersedes prior insights (Table 3.6)
POST /api/calls/:id/cancel	Cancel queued/running call	Idempotent; sets CANCELED

Table 5.3 - Calls and ingestion

Method / Path	Purpose	Contract notes
GET /api/calls/:id/transcript	Full transcript + segments	Segments ordered by idx; includes speaker labels and ms timings
GET /api/calls/:id/segments/:idx/context	Segment with neighbors	For evidence links opening in context (window 2 before/after)
GET /api/calls/:id/insights	Insights for the call	filter: status, type; includes superseded when ?history=1
PATCH /api/insights/:id	Confirm or dismiss	status: CONFIRMED or DISMISSED; confirmed MEDDIC writes through to Deal/Contact flags
GET /api/calls/:id/scorecard	Scorecard with dimensions	Includes evidence links per dimension (Ch. 3 rubric)
GET /api/calls/:id/runs	AI provenance panel	ProviderRun rows incl. fallback chain and cost (FR-023)

Table 5.4 - Transcript, insights, and review

Method / Path	Purpose	Contract notes
GET /api/deals	List deals	filters: stage (multi), archived, q; sorted by healthScore desc default
POST /api/deals / PATCH /api/deals/:id	Create / update deal	Stage changes bump lastActivityAt and append timeline
DELETE /api/deals/:id	Soft-delete (archive)	Sets archivedAt; hard delete is a separate explicit call with warning payload
GET /api/deals/:id/timeline	Auto-built timeline (FR-012)	Mixed entries: calls, insights, stage changes, notes; cursor by occurredAt
GET /api/pipeline	Board payload (FR-011)	Deals grouped by stage with risk badges and health; one call, board-ready
GET/POST/PATCH/DELETE /api/contacts	Contact CRUD	Contact is deal-scoped; champion/EB flags writable here

Table 5.5 - Deals, contacts, and pipeline

Method / Path	Purpose	Contract notes
GET /api/coaching/trends	Trend series (FR-015)	Query: window (days), dimension; returns per-call scores + rolling mean; 200 with empty series before 3 calls
GET /api/coaching/best-practices	Library list (FR-016)	filters: category, status, tag, q
POST /api/best-practices	Promote or accept	body: fromSegmentIds (USER_PROMOTED) or accept a MODEL_SUGGESTED candidate
POST /api/library/search	Semantic search (FR-018)	body: {q, filters: {dealId, contactId, from, to, speaker}}; returns hits with snippet, score, source link; grouped view via ?group=call
GET/POST /api/calls/:id/notes, PATCH/DELETE /api/notes/:id	Notes CRUD	AI-origin notes are drafts until first edit (Table 4.9)
GET /api/calls/:id/follow-up	Fetch draft	Single active draft per call
PATCH /api/follow-ups/:id	Edit, mark COPIED/SENT/DISCARDED	sentAt stamped on SENT

Table 5.6 - Coaching, library, notes, and follow-ups

Method / Path	Purpose	Contract notes
GET /api/jobs	Job monitor feed	Active + recent jobs with attempts, lastError
GET /api/events	SSE stream (UI live updates)	text/event-stream; events in Table 5.8; heartbeat every 15 s
GET /api/providers/health	Provider health cards (FR-021)	Per provider: up, latencyMs, models[], lastError; 15-min error cache
POST /api/providers/test	One-click provider test	Runs a trivial task on the chosen provider; 429 throttled to 1/10 s
GET /api/budget	Spend dashboard (FR-022)	month, cap, spent, entries[]; 402 never returned here - this is read-only
PATCH /api/settings	Update a Setting key	Whitelisted keys only (Ch. 11); budget.monthlyCapUsd change is logged
GET /api/system/health	Liveness for monitors	{ db: ok, media: {bytesFree}, ollama: bool, queueDepth }
POST /api/system/backup	Trigger backup now (Ch. 10)	Returns the backup file path; 202, runs async
GET /api/system/export	Export JSON bundle (FR-024)	Everything except media bytes; respects RedactionRule scope ALL

Table 5.7 - Jobs, events, providers, budget, and system

Server-sent events: the live update contract

The UI subscribes to one SSE stream and reacts to typed events; there is no polling anywhere in the app. Events carry the minimum payload needed to decide whether to refetch (IDs and statuses, not entities), which keeps the stream chatty-cheap and the client logic trivial: on event, invalidate the named resource in the client cache and refetch if visible. The connection auto-reconnects with backoff; missed events are recovered by the refetch-on-reconnect handler, which is safe because events are hints, not sources of truth.

Event	Payload	UI reaction
call.stage	{callId, from, to, at}	Update progress rail; refetch call header if visible
call.failed	{callId, stage, errorCode}	Error card on inbox row; toast if the call is open
call.ready	{callId}	Toast "Call ready"; refetch inbox and any open detail
insight.created	{callId, insightId, type}	Badge count on review queue
job.updated	{jobId, status, attempts}	Job monitor row update
provider.health	{provider, up, latencyMs}	Provider status pill update
budget.warning	{spentUsd, capUsd, percent}	Footer spend meter color
heartbeat	{}	Connection liveness only

Table 5.8 - SSE event catalog

6 · AI Engine and Prompt Specification

This chapter specifies the hybrid AI engine at the level a builder needs: the twelve tasks, each with its input, output schema, model policy, quality gate, and cost; the routing decision procedure that picks local-first with enumerated escalation; the retry ladder; the budget ledger that makes the cost cap real; the redaction pipeline that guards the egress contract; and the evaluation harness that turns prompt tinkering into science. v1 established the policy language; this chapter makes it executable. Every rule here is implementable as written - where a judgment call is unavoidable, the default is stated and the knob is named.

Task registry

Twelve tasks cover every AI interaction in AMBER. T01 and T10 are permanently local by policy (audio never leaves; embeddings are cheap and local is faster than the network). The remaining ten are text tasks that follow the routing policy. Each task carries a modelHint that maps to the catalog in Chapter 11; hints are small (7-8B class), medium (13-20B class), or large (cloud frontier), and the catalog resolves hints to concrete model ids, so a new model is a config edit, never a code change.

ID	Task	Class	Default route	Quality gate
T01	Transcribe (STT)	local-fixed	whisper small (medium if RTF allows)	Segment coverage at least 95% of speech; word count sanity vs duration
T02	Speaker relabel	small	Ollama small	Only REP/PROSPECT/UNKNOWN labels emitted; no empty labels
T03	Call summary	small	Ollama small, escalate medium	5-8 sentences; mentions at least 2 distinct call topics; schema valid
T04	MEDDIC extract	medium	Ollama medium, escalate cloud	All 6 fields present (NOT_FOUND allowed); evidence ids valid; F1 at least 0.85 on golden set
T05	Risk and signal detect	medium	Ollama medium, escalate cloud	Every flag has evidence segment id; no duplicate flags per type
T06	Scorecard (rubric)	medium	Ollama medium, escalate cloud	Six dimensions 0-10; each with evidence; arithmetic fields recomputed, not trusted from model
T07	Follow-up draft	small	Ollama small	Subject under 80 chars; body references at least 1 concrete detail; tone knob respected
T08	Notes draft	small	Ollama small	Action items are imperative sentences; open questions are questions
T09	Redact (regex + NER)	local-fixed	regex pass + local NER (small)	Zero known-pattern leaks in redaction test suite (Ch. 9)
T10	Embed text	local-fixed	nomic-embed-text	Vector dim matches model; cosine sanity on fixture pair
T11	Library answer (RAG)	medium	Ollama medium	Every claim cites a hit id; refuses when top score below threshold
T12	Best-practice suggest	small	Ollama small	Suggestion links real segment ids; category from fixed list

Table 6.1 - Task registry T01-T12

Per-task specification and output schemas

The four tasks below are specified in full - input contract, prompt shape, and output schema - because they carry the most product value and the most risk. The remaining tasks follow the identical pattern; their prompt texts ship as the seeded PromptTemplate rows (one active version each, editable in Settings per NFR-14), and their schemas live in the shared zod directory next to the API schemas. All prompts share three standing rules, baked into every system prompt: answer only from the provided transcript; never invent a segment id - use only the numbered ones given; and emit strict JSON matching the schema with no commentary.

Aspect	Specification
Input	Segments with idx + speaker + text; deal context (title, stage, known MEDDIC values); language hint
System prompt shape	Sales-call qualification analyst; defines each MEDDIC field operationally per Table 3.3; NOT_FOUND is a valid answer; confidence rubric 0-1; evidence must quote segment idx
Output schema	{fields: [{key: M|E|D_CRITERIA|D_PROCESS|I|C, value: string|null, notFound: bool, confidence: float, evidenceIdx: [int], quote: string}]}
Post-processing	evidenceIdx mapped to Segment ids; confidence below 0.5 -> UI weak-signal; confirmed C/E flags write back to Contact (champion / economic buyer)
Escalation triggers	Schema invalid after 1 repair-retry; any field confidence below 0.3 on a NEGOTIATION-stage deal; transcript over 24k chars (context overflow)
Cost profile	Local medium: $0; cloud fallback: approx. $0.004-0.01 per 30-min call at 2026 mid-tier pricing

Table 6.2 - T04 MEDDIC extraction: full specification

Aspect	Specification
Input	Segments; the active rubric definition (Setting rubric.v1 - dimension names, weights, full-marks text from Table 3.5); arithmetic metrics from S5 (talk ratio, question count, longest monologue) passed as facts
System prompt shape	Fair, evidence-citing coach; scores 0-10 per dimension against the rubric text; must cite segment idx for every judgment; talk balance scored against the 40-60% target given as fact, not re-estimated
Output schema	{dimensions: [{key, score: int 0-10, evidenceIdx: [int], comment: string}]} - weights applied in code, never in the model
Post-processing	Weighted overall computed in code from rubric weights; arithmetic fields written from S5 facts, overwriting any model echo
Escalation triggers	Schema invalid after repair-retry; any dimension missing; score spread of 0 across all dimensions (suspicious flatness)
Cost profile	Local medium $0; cloud fallback approx. $0.003-0.008 per call

Table 6.3 - T06 Scorecard: full specification

```
T03 summary - seeded system prompt (active v1):
You are a precise analyst reading one sales call transcript.
Rules: (1) Use only the transcript. If information is absent, say so plainly.
(2) Cite evidence as [seg:N] using the given segment numbers; never invent ids.
(3) Output strict JSON only: {"summary": string (5-8 sentences),
  "topics": [{"name": string, "segmentIdx": [int]}] (2-5 topics),
  "nextStepSuggested": string|null} (4) The summary must state what the
prospect wants, what was agreed, and what remains open - in that order.
```

T03 prompt, seeded version. Placeholders {{segments}} and {{language}} are filled by the engine; the JSON shape is the quality gate.

Routing policy and the retry ladder

Routing is a pure function of the task policy, the payload, and the live state of providers and budget - no LLM decides routing, and no stage knows providers exist. The decision procedure runs per task, in order: (1) policy tier - local-fixed tasks stop here; (2) payload fit - if input exceeds the local model context window or minContextChars, escalate; (3) budget - cloud is allowed only if the ledger has headroom (hard stop otherwise); (4) provider health - skip providers marked unhealthy in the last 15 minutes; (5) preference order from models.yaml. The retry ladder for a failed attempt is exactly: repair-retry once on the same provider (schema failures only), then escalate one step up the preference chain, then mark the task failed with errorCode and surface it per Table 3.6. Cloud-to-cloud fallback counts against the same ladder.

Trigger	Action
Output fails JSON schema after 1 repair-retry locally	Escalate to next preference (medium model, then cloud)
Input exceeds local context window (measured, not guessed)	Escalate directly to a long-context cloud model
Quality gate fails on substance (missing fields, flat scores)	Escalate once; if cloud also fails, task fails honestly
Cloud key invalid / provider 4xx-5xx	Skip provider 15 min; try next; if none, degrade to local where policy allows, else fail
Ledger cap would be exceeded by this run	Refuse cloud (BUDGET_EXCEEDED); try local; if local cannot serve the tier, park the task
User sets ROUTING_MODE=LOCAL_ONLY in Settings	Cloud legs removed for all non-fixed tasks until reverted

Table 6.4 - Escalation triggers, enumerated (no judgment calls at runtime)

Budget ledger mechanics

The monthly cap is enforced at the only place it cannot be bypassed: the transaction that records spend. Before any cloud call, the engine reads the month row set (BudgetEntry for YYYY-MM) in a transaction, sums running totals, and refuses the run if projected spend (estimate from token pricing in Setting prices.*) exceeds the cap; on success it writes the entry and the OK status in one commit. The estimate is conservative (max tokens for the tier), so the cap is never breached by a slow-finish run. Local runs write nothing - zero-cost tasks have zero ledger entries, which keeps the spend chart honest. The footer meter (Ch. 7) reads the same table, so what you see is what the engine enforces.

Redaction pipeline

Redaction runs on every text payload before it can cross the network boundary, in rule priority order: REGEX rules first (fast, deterministic), then the local NER pass for PERSON, ORG, AMOUNT entities when enabled. MASK wraps the value in typed brackets ([EMAIL]), REPLACE substitutes a fixed token, SKIP drops the containing sentence entirely (used for the nastiest patterns, like card numbers, where even masked context is unwanted). The redacted payload, not the original, is what the cloud adapter ever receives; the adapter interface takes pre-redacted text by construction (R09's test asserts this at the adapter boundary). The Settings test pad runs any text through the pipeline and shows the diff, because a rule you cannot see working is a rule you cannot trust.

Prompt versioning and the evaluation harness

Prompts are data (NFR-14): every LLM task has exactly one active PromptTemplate version, edits create a new version, and activation is explicit. That is what makes the harness meaningful: a version is born, run against the golden set, and promoted only if it beats or matches the incumbent. The harness (POST /api/evals/run in the Should tier; CLI in v1 Must via npm run eval) executes every EvalCase for a task against a chosen provider/model/prompt-version triple, scores it with the task metrics below, writes EvalRun rows, and prints a comparison table against the current active combination. The regression gate: a prompt or model change that drops any task metric below its threshold cannot be activated; the UI enforces it, the CLI respects it with a --force flag for experiments that are then marked experimental.

Task	Metric	Threshold
T03 Summary	Topic recall vs expected topics (judged by embedding similarity at 0.8) + length band compliance	At or above 0.80; 5-8 sentences
T04 MEDDIC	Field-level F1 on value extraction; evidence precision (cited idx correct)	F1 at or above 0.85; evidence at or above 0.90
T05 Signals	Flag-level precision and recall vs expected flags	F1 at or above 0.80
T06 Scorecard	Dimension score MAE vs human baseline; evidence validity	MAE at or below 1.5; evidence 1.00
T07 Follow-up	Concrete-detail recall (does the draft cite real facts); tone classifier pass	At or above 0.90; tone pass
T09 Redaction	Known-pattern leak count (must be zero); over-redaction rate	0 leaks; over-redaction at or below 5%
T11 Library answer	Citation validity; refusal correctness on no-hit cases	1.00 valid; refusal correct on all fixtures

Table 6.5 - Eval harness: metrics and promotion thresholds per task

7 · UX and Screen Specification

This chapter turns the Warm Calm design language into a buildable surface: the tokens, the component kit, the twenty screens with their layouts and states, the interaction rules, and the keyboard layer. The design intent stays what v1 set: cream backgrounds, generous radius, one warm accent, typography that is calm at rest and legible in long transcript sessions. Two product-level UX laws govern everything: never show a number the system cannot explain (every metric links to its provenance), and never let a waiting state look broken (every async surface has a designed state for empty, loading, error, and done).

Design tokens

Tokens are CSS variables emitted once (globals.css) and consumed by every component; no component hard-codes a color, radius, or shadow. The palette is the cascade system from v1 mapped to UI roles. Type uses Inter for UI and Noto Serif for long-form reading (transcripts read better in a serif at length); both are self-hosted to keep the app fully offline-capable. Spacing is a 4-point scale; radius is deliberately large (12-16px) because warmth is partly geometry.

Group	Token	Value / rule
Surface	bg-page / bg-card / bg-raised	#f7f4ef / #fffdf9 / #f1ece4 - cream stack, page darkest at rest
Ink	ink-primary / ink-secondary / ink-muted	#2b2622 / #5a5048 / #94897d
Accent	accent / accent-soft / accent-ink	#d76932 (amber) / #f6e3d5 (tint fills) / #8a4520 (text on tint)
Structure	border / border-strong	#e5dcd0 / #cdbfae
Semantic	positive / warning / danger	#4e7d51 / #b37a2e / #a84a3a - muted, same saturation family
Type	font-ui / font-reading / font-mono	Inter / Noto Serif / Sarasa Mono
Type scale	display / title / body / small / micro	28 / 20 / 15 / 13 / 11 px
Radius	r-sm / r-md / r-lg / r-pill	8 / 12 / 16 / 999 px
Spacing	sp-1..sp-6	4 / 8 / 12 / 16 / 24 / 32 px
Shadow	shadow-sm / shadow-md	soft warm shadows, y-offset 1/2, blur 3/8, 6-8% ink

Table 7.1 - Warm Calm token set (single source of truth)

Screen inventory

Twenty screens cover v1. Each row names the screen, its job, its primary components, and the states it must design (E=empty, L=loading, R=error, plus special states noted). Screens 4, 7, 8, 11, and 15 receive detailed layouts in the next section because they carry the most complexity; the rest follow the component kit and standard page scaffold (max-width 1200, 24px gutters, sticky left rail).

#	Screen	Job	Components	States
1	Dashboard	Today at a glance: recent calls, pipeline snapshot, review queue, spend meter	Stat cards, call list, mini board, queue badge	E L R
2	Inbox (Calls)	All calls, status, filters, bulk archive	Table, status chips, filter bar, dropzone	E L R
3	Upload	Drop or pick file, set metadata	Dropzone, metadata form, progress rail	L R (415 message)
4	Call detail	Read everything about one call	Tabs: Overview / Transcript / Insights / Scorecard / Notes / Follow-up / Provenance	E L R + stage rail
5	Transcript reader	Read, search in-call, highlight, promote	Serif reader, speaker rail, find bar, selection popover	L + no-transcript state
6	Insight review	Confirm or dismiss extracted insights	Card queue, evidence drawer, bulk actions	E L + zero-active state
7	Pipeline board	See and move deals	Kanban columns, deal cards with badges, drag	E L R + drag ghost
8	Deal detail	One deal, full story	Header (health, stage), MEDDIC panel, timeline, contacts, calls list	E L R
9	Contacts	Browse contacts per deal	Table, flags chips	E L
10	Coaching home	Trends and rubric breakdown	Line charts, dimension bars, top strengths/gaps	E (calibration) L
11	Scorecard detail	One call, coached	Six dimension cards with evidence, compare-to-top-quartile	L
12	Best practices	Your playbook	Card grid, search, category filter	E L
13	Library search	Find anything said	Search bar, filters, hit list with snippets, group toggle	E (tips) L + no-hits
14	Follow-up editor	Edit and track the draft	Subject/body editor, tone select, copy button, status	L
15	Settings - Providers	Health, catalog, test	Provider cards, model list, test button	L R + unhealthy state
16	Settings - Budget	Cap and spend	Cap input, spend bar, entry table	L
17	Settings - Redaction	Rules and the test pad	Rule list with toggles, priority, diff pad	L

#	Screen	Job	Components	States
18	Settings - Prompts	View and version prompts	Task list, version diff, activate with eval gate	L R (gate fail)
19	Job monitor	What the worker is doing	Job table, retry buttons, error detail	E L
20	Data and backup	Export, backup, retention	Export button, backup list, restore upload	L R

Table 7.2 - The 20-screen inventory

Detailed layouts for the five complex screens

Call detail (4) is the app's center of gravity and uses a fixed scaffold: a 56px header strip (title, deal chip, stage rail with six pips, actions), then a tab body. The stage rail is the state machine made visible - pips for UPLOADED through READY light progressively amber, the active stage pulses gently, and a FAILED pip turns danger-red with a retry affordance on hover. Overview tab is a 12-column grid: summary card full-width, then MEDDIC six-pack (2x3 cards, each with value-or-NOT_FOUND, confidence dots, evidence links), risk flags row, metrics row (talk ratio dial, questions, monologue), then notes and follow-up side by side. Evidence links everywhere open the transcript reader scrolled to the segment with a soft highlight pulse - this single interaction is the most-used in the app and gets a 200ms ease, not an instant jump.

Pipeline board (7) renders stage columns LEAD through WON/LOST with PARKED collapsed to a rail; cards show account, title, value, health dot (green/amber/red by 66/33 cutoffs), and up to two risk badges. Drag uses a warm ghost (accent-soft, 90% scale); drop updates optimistically and reconciles with the PATCH response. Deal detail (8) leads with the health gauge and stage stepper, then the MEDDIC completeness panel (six cells, filled cells in accent-soft), the auto-built timeline down the left column, contacts with champion/EB flags on the right, and the deal's calls as compact rows. Scorecard detail (11) shows six dimension cards (score dial, evidence links, coach comment) with the weighted overall on top and a compare toggle overlaying your top-quartile means as ghost markers. Settings - Providers (15) is one card per adapter: status pill (live health), model dropdown sourced from models.yaml, latency of the last test, and a test button whose result animates in place.

Interaction rules and keyboard layer

Three interaction rules keep the app fast and legible. Optimistic UI for metadata edits (title, stage, insight status) with rollback on error; never for pipeline-affecting actions (reanalyze, retry), which show a confirming state. Progressive disclosure: every list row expands in place rather than navigating where possible. Live updates via SSE only - no polling - with toasts capped at three stacked and a dedicated activity drawer for overflow.

Keys	Action
g then c / d / k / l / s	Go to Calls / Deals / Coaching / Library / Settings
/	Focus search (Library from anywhere; in-call find when transcript open)
j / k	Next / previous row in the active list (inbox, review queue, search hits)
e / a	Archive selected call / accept shown insight (review queue)
x / v	Dismiss shown insight (x) / confirm MEDDIC value to deal (v)
r	Retry failed call (when a failed row is selected)
?	Shortcut cheat sheet overlay

Table 7.3 - Keyboard shortcuts (global unless noted)

Accessibility notes are short because the audience is one, but they are also free wins: all interactive elements carry visible focus (2px accent outline), color is never the only signal (risk badges pair icon + text), the transcript reader supports a dyslexia-friendly toggle (font bump plus wider leading), and all charts ship with a table view toggle. Empty states teach rather than scold - the review queue's empty state explains what insights are and links to the next unreviewed call; the coaching screen's calibration state explains why three calls are needed before trends mean anything (FR-015).

8 · Security, Privacy and Data Management

AMBER's security posture is unusual and unusually strong in one dimension: the most sensitive asset - raw audio of real customers - never crosses a network boundary, by construction. The rest of the posture is proportionate to a single-user local application that optionally sends redacted text to cloud APIs. This chapter enumerates the threats worth thinking about, answers each in design, classifies the data, fixes key management, states the egress contract as a testable invariant, and defines backup, restore, and retention. Nothing here adds friction to daily use; the goal is that privacy is a property of the architecture, not a discipline of the user.

Threat model

The model below is deliberately compact: ten threats, each with the design answer and where that answer is verified. It excludes organizational threats (insiders, multi-tenant bleed) because they structurally cannot exist in a single-user local app, and it treats "attacker with local code execution" as out of scope - that attacker owns the machine and every application on it. What remains is the realistic set for this deployment shape.

#	Threat	Design answer	Verified by
T1	Audio bytes reach a cloud provider	STT is local-fixed at the code-path level; cloud adapters accept text only, typed as such	Integration test asserting request payloads (Ch. 9, NFR-05)
T2	Sensitive text leaves un-redacted	Redaction pipeline wraps every outbound text transform; adapters receive pre-redacted text by interface	Adapter boundary test + redaction suite (T09)
T3	API keys leak via DB, logs, or exports	Keys live only in .env; never written to any table, log line, or export path; export bundle is key-free by schema	Export snapshot test + log scrubber test
T4	Local session hijack from another tab/app	Passphrase-gated session cookie, HttpOnly, SameSite=Lax, 12-hour expiry, bound to loopback use	Auth middleware unit tests
T5	Malicious upload (zip-bomb audio, giant file)	Size/duration rules at S1 (FR-001); ffmpeg runs with resource limits; content is probed before decode	Upload validation tests
T6	Prompt injection via transcript content	Transcript is data, never instructions; system prompts state this; output schemas constrain; evidence ids validated post-hoc	Eval fixture with adversarial transcript
T7	Cloud provider stores or trains on text	Documented per-provider data-retention flags in models.yaml; providers without zero-retention options can be excluded by policy	Provider config check + runbook note
T8	Data loss (disk failure, accidental delete)	Scheduled VACUUM INTO backups + media sync; restore rehearsed; soft-deletes for deals and calls	Backup/restore procedure dry-run (Ch. 10)
T9	Supply-chain compromise (npm/PyPI dep)	Lockfile-only installs; renovate-style update cadence; no postinstall scripts from unvetted deps; minimal dep tree	npm audit in pre-push checks
T10	Screen shoulder-surfing / shared machine	Passphrase lock returns in 12h or on demand; no sensitive data in page titles; session expires on browser close (optional knob)	Manual check in runbook dry-run

Table 8.1 - Threat model: ten threats, each answered in design

Data classification and the egress contract

Four classes cover every byte in the system. Class A (audio) - original and normalized recordings: never leaves the machine, never enters an export, stored content-addressed in MEDIA_DIR. Class B (raw conversation text) - transcripts and segments: leaves the machine only through the redaction pipeline and only when a task's policy allows cloud. Class C (derived intelligence) - insights, scorecards, summaries, drafts: same egress rules as B (they quote the conversation). Class D (operational metadata) - deals, contacts, settings, logs, spend: never sent to providers at all; cloud prompts receive the minimum deal context the task needs (title, stage) and nothing else. The egress contract is the summary invariant: A never egresses; B and C egress only redacted and only per policy; D stays home.

Key management

Keys are environment variables in .env, loaded once at boot into a frozen config object; nothing downstream re-reads the environment. The file is chmod 600, git-ignored by the scaffold (with a committed .env.example documenting every variable), and never logged - the logger scrubs any string matching known key shapes as a belt-and-braces measure. Rotation is documented per provider in the runbook (replace value, restart, verify with the provider test button); no key rotation requires a database change because no key is ever persisted (NFR-13). The session passphrase is an Argon2id hash in .env, set by a first-run prompt, so even the passphrase itself is not stored in clear.

Backup, restore, and retention

Backups are boring by design: a nightly job runs SQLite VACUUM INTO to a dated file in BACKUP_DIR and rsyncs the media vault (hard-linked where the filesystem allows, so dedup survives backup). The runbook (Ch. 10) contains the exact commands and the restore drill - restoring is copying the backup database over dev.db and the media directory back into place, then running the integrity check endpoint. Retention keeps everything by default: audio is the irreplaceable asset and personal scale will not fill a disk for years (1,000 calls at ~30 MB normalized is ~30 GB). The one scheduled cleanup is cloud-side by nature: nothing - AMBER retains no cloud state. Exports (FR-024) are the user-controlled offboarding path: one JSON bundle with every class B-D datum, redacted per rule scope ALL, and a manifest of media file hashes so the bundle plus the media dir fully reconstructs the system.

Datum	Where it lives	Backed up	Retention	Egress
Original + normalized audio	MEDIA_DIR, content-addressed	Nightly sync	Forever (default)	Never
Transcripts + segments	SQLite	Nightly VACUUM INTO	Forever	Redacted, per policy
Insights, scorecards, drafts	SQLite	Nightly	Forever	Redacted, per policy
Deals, contacts, notes	SQLite	Nightly	Forever	Never (min context in prompts)
Provider runs + spend	SQLite	Nightly	Forever (audit trail)	Never
API keys	.env only	Excluded by policy	Until rotated	Never
Session records	In-memory + cookie	No	12 hours	Never

Table 8.2 - Data management policy summary

One legal note, kept proportionate: recording-consent law varies by jurisdiction and by the other party's location, and it is the operator's responsibility, not the software's. AMBER's design choices reflect good practice - post-call upload means the operator consciously selects what enters the system, and nothing is captured silently. If your calls span jurisdictions with all-party consent rules, the honest path is verbal notice at the top of the call; the follow-up template in Settings includes an optional one-line recording disclosure for exactly this reason.

9 · Testing and Quality Plan

The test strategy protects the three things that would be catastrophic to break: the privacy invariant (audio never egresses), the pipeline's crash safety, and the AI layer's output discipline. Everything else gets proportionate coverage. The plan is sized for one builder: fast unit tests that run on every save, integration tests on pre-push, an end-to-end happy path that runs nightly and before every phase acceptance, and the eval harness guarding every prompt or model change. Coverage targets are set where they mean something (core logic) and not chased where they do not (UI chrome).

Test pyramid and tooling

Vitest covers unit and integration (it runs the route handlers in-process against a temp SQLite file); Playwright covers the E2E surface against a booted dev server with mocked providers; the eval harness is a plain Node CLI writing EvalRun rows. The fixtures directory is the shared substrate: synthetic audio generated at test time (ffmpeg sine/silence composites with known scripts), golden transcripts with hand-annotated expected extractions, and provider mocks that replay canned responses. Everything runs offline - cloud adapters are always mocked in automated tests; the only live-cloud code path is the manual provider test button.

Layer	Scope	Runtime	Runs on
Unit	Routing decision function, redaction rules, ledger math, health score, state machine transitions, metrics arithmetic, zod schemas	Under 10 s	Every save (watch mode)
Integration	API routes against temp DB, worker stages with fixture audio, adapter mocks, SSE stream, upload rules	Under 60 s	Pre-push hook
E2E	Upload-to-ready happy path, board drag, review queue flow, search, settings flows	3-5 min	Nightly + phase acceptance
Eval harness	Golden set per task against active or candidate prompts/models	Minutes per task	Every prompt/model change (gate)
Performance	RTF benchmark, API p95, search latency at 1k synthetic calls	5 min	Phase 1, 4 acceptance + monthly

Table 9.1 - Test pyramid: layers, scope, and when they run

Fixtures: the golden set

The golden set is ten transcripts chosen to span the difficulty space, each with expected outputs for the tasks that consume it. They are committed files (no live generation), and the harness treats them as immutable - improving the golden set is an explicit, reviewed act, because it is the yardstick everything else is measured against. The three synthetic audio fixtures exist for pipeline tests, not quality measurement: their scripts are known, so transcription correctness is checkable mechanically.

#	Fixture	Why it exists	Tasks scored
G1	Clean discovery call, 25 min, two speakers	Baseline happy path	T03 T04 T05 T06 T07
G2	Pricing negotiation with objections	Pricing friction, objection handling	T04 T05 T06
G3	Technical demo, three speakers	Multi-speaker relabel, jargon	T02 T03 T04
G4	Short check-in, 6 min, no next step	NO_NEXT_STEP flag, short-context	T03 T05
G5	Heavily accented prospect, noisy line	The R01 case - transcription robustness	T02 T04
G6	Renewal call with existing champion	Champion detection, MEDDIC carry-over	T04 T05
G7	Call with competitor comparisons	COMPETITOR_MENTIONED	T05
G8	Multilingual call (EN + DE mix)	Language hint handling	T01 T03
G9	Adversarial: prompt injection in transcript	T6 defense verification	T03 T04 T06
G10	Redaction torture: emails, IBANs, card-like numbers, names	T09 leak check	T09

Table 9.2 - Golden set composition

Coverage matrix and quality gates

The matrix maps requirement groups to the layer that verifies them, so a missing test is visible as a hole in the grid rather than a feeling. The two named invariants get dedicated tests that fail the build loudly: the egress invariant (a spy around every adapter's network call asserts no payload contains fixture audio bytes or un-redacted Class B fixtures) and the crash-safety invariant (the E2E harness SIGKILLs the worker at each stage boundary and asserts clean resumption). AI quality gates are the Table 6.5 thresholds; the project-level gate for phase acceptance is that all FR acceptance criteria pass at least once in the phase's E2E run and the eval harness is green on the active prompt set.

Requirements	Verified by	Notes
FR-001..002 (upload, dedup)	Integration: upload rules, SHA-256 link	Table-driven format/size cases
FR-003 (RTF budget)	Performance benchmark vs threshold	Reference-machine measured, env-stamped
FR-004..005 (transcript, metrics)	Integration + unit arithmetic check on G1	Hand-computed talk ratio fixture
FR-006..007 (summary, notes, draft)	Eval harness on golden set	Thresholds per Table 6.5
FR-008 (crash safety)	E2E SIGKILL drill at each stage	Also asserts no double-run on resume
FR-009..013 (MEDDIC, flags, board, timeline, health)	Eval (F1) + integration (board, timeline assembly) + unit (health formula)	Formula unit tests pin Table 3 health v1
FR-014..017 (coaching)	Eval (MAE) + E2E (trend screen) + unit (weighted scoring)	
FR-018..020 (library, search, filters)	Integration search suite + perf at 1k calls	Synthetic corpus generator
FR-021..023 (providers, budget, logging)	Integration: health, test button, ledger cap refusal, run rows	Cap refusal test uses 0.01-dollar cap trick
FR-024 + NFR-05 (redaction, egress)	Adapter-boundary spy + T09 suite on G10	Build-failing invariants
NFR-01..02, 12 (latency, startup)	Performance suite	p95 sampler on the integration run
NFR-10 (backup/restore)	Runbook dry-run checklist (Ch. 10)	Manual, scripted, dated

Table 9.3 - Requirements-to-tests coverage matrix

Pre-commit runs lint, format check, unit tests, and the egress invariant; pre-push adds integration; the nightly job adds E2E and a database integrity check. A failing eval gate does not block commits - it blocks activation of the prompt or model change that caused it, which is the difference between a quality gate and a productivity trap. Test data never leaves the machine; cloud mocks make that structural.

10 · Operations and Deployment Runbook

This runbook covers the full life of a running AMBER install: prerequisites, install, first-run, model provisioning, upgrade, backup and restore, monitoring, and a troubleshooting playbook written as symptom, diagnosis, fix. It assumes a single machine owned by one person, which means every procedure fits on a screen and none requires a second terminal. The dry-run at the end is a scripted rehearsal - running it after Phase 5 acceptance is the operational definition of done for the project (Ch. 12).

Prerequisites and install

Requirement	Minimum	Recommended
OS	macOS 14 (arm64) or Ubuntu 22.04 (x64)	Same - these are the tested pair
RAM	8 GB	16 GB (comfortable headroom for 8B models)
Disk	10 GB free	40 GB (models + a year of audio)
Node.js	22 LTS	Managed via nvm
Ollama	Installed and running	Latest stable
ffmpeg	Bundled via ffmpeg-static	System ffmpeg only if you prefer
GPU	None required	Any - Ollama uses it automatically

Table 10.1 - Prerequisites

Install is seven commands, and the scaffold is designed so that the seventh ends with a working, empty app: clone the repository; install dependencies with the lockfile (corepack enable and pnpm install --frozen-lockfile); copy .env.example to .env and set the session passphrase; run prisma migrate deploy to create dev.db; run the seed (creates default settings, starter prompt templates, redaction rules, and fixture stubs); start the app (pnpm dev, or pnpm build and pnpm start for daily use); open localhost:3000 and complete the first-run wizard below. Total time on a fresh machine: about ten minutes, plus model pulls.

First-run wizard and model provisioning

The first-run wizard is four steps and exists to make the prerequisites visible rather than assumed: (1) session passphrase set and confirmed; (2) provider health check - Ollama detected, cloud keys optional and individually skippable; (3) model provisioning - a button per required model that runs the right pull and shows progress (the catalog in Ch. 11 lists exact tags: whisper is bundled with the app, qwen3:8b, gpt-oss:20b if you have headroom, nomic-embed-text); (4) the calibration pass - the wizard runs the eval harness on the golden set to establish your machine's baseline RTF and quality numbers, which become the reference point for every future change. Skipping step 4 is allowed but the Settings screen keeps a gentle nag until it runs once.

Upgrade procedure

Upgrades follow the same shape as installs because the data layer is designed for it: git pull; pnpm install --frozen-lockfile; prisma migrate deploy (additive-first per Ch. 4 policy); ollama pulls re-checked against the catalog (new model tags appear as suggestions, never auto-pull); pnpm build; restart. Before the migrate step, the updater takes an automatic safety backup (VACUUM INTO with a pre-upgrade tag) - a failed migration restores from it and stops, rather than half-upgrading. After any migration that bumps the schemaFeature flag, the eval harness runs automatically and reports; a red eval report after upgrade pins the cause to a schema or model change, not to your imagination.

Backup, restore, and monitoring

Nightly backup is a scheduled job (cron or launchd, both templates shipped): VACUUM INTO BACKUP_DIR/amber-YYYYMMDD.db plus rsync --link-dest of MEDIA_DIR; the job keeps 30 daily snapshots and prunes older ones. Restore is the drill that must be rehearsed, not read: stop the app; move the current dev.db aside; copy the chosen snapshot to dev.db; rsync the matching media snapshot back; start the app; open /api/system/health and confirm db ok plus queueDepth; spot-check one call's transcript and audio. The runbook dry-run performs this end-to-end on a scratch directory. Monitoring is deliberately minimal: the health endpoint (db, disk, Ollama, queue depth) polled by a tiny menu-bar widget or a cron that writes to a log; provider health lives in the app itself (FR-021); and spend is on every footer (FR-022). There is no telemetry because there is no one to phone home to.

Troubleshooting playbook

The playbook below covers the failures a real install produces. The general diagnosis order: check /api/system/health first (it names the layer), then the job monitor (queue state), then provider health (cloud or Ollama), then logs (stdout in dev; the log file in production). Every row ends with either a fix or the honest state the system reaches when the fix is not applicable.

Symptom	Diagnosis	Fix
Upload stuck in UPLOADED	Worker not polling - check job monitor and health queueDepth	Restart app; stale job locks are reclaimed on boot by design
NORMALIZE_FAILED on valid file	ffmpeg probe says codec unsupported or file truncated	Re-encode externally (ffmpeg -i in -ac 1 -ar 16000 out.wav) and re-upload; file kept for inspection
TRANSCRIBE_EMPTY on real call	Audio silent or level too low pre-normalization	Check waveform in any player; re-normalize with stronger loudness and retry the stage
Transcription crawling (RTF over 1)	Model too large for machine, or CPU-only with medium model	Settings: switch whisper tier to small; or accept slower processing; RTF is shown per call
Ollama 403/500 on model pull	Tag moved or disk full	Re-pull exact tag from models.yaml; free disk; catalog is config (R08)
ANALYZE_FAILED with PROVIDER_UNAVAILABLE	Ollama not running and no cloud keys or budget	Start Ollama (or configure a cloud key); retry the stage - partials preserved
BUDGET_EXCEEDED mid-month	Cloud usage above cap	Raise cap knowingly, or set ROUTING_MODE=LOCAL_ONLY until reset; spend table shows what cost what
Cloud 401 from one provider	Key revoked or expired	Replace key in .env, restart, use provider test; other providers unaffected
Search returns nothing after model switch	Embeddings stale - vectors from old model	Settings: run re-index job; progress on job monitor; search degrades to substring until done
App slow to start	Large dev.db or media integrity scan	One-time; if recurrent, run the backup-optimize pass (VACUUM) and check disk space
dev.db locked error	Second app instance running	Close the other instance; SQLite single-writer rule (by design, NFR scope)
Scores changed after upgrade	Prompt or rubric version activated	Check Settings-Prompts version history; eval report shows the delta - this is provenance working, not a bug

Table 10.2 - Troubleshooting: symptom, diagnosis, fix

11 · Configuration Reference

Configuration lives in exactly two places, and the boundary between them is a rule, not a habit. The environment (.env) holds infrastructure and secrets: paths, keys, routing mode, thresholds that change the machine's behavior. The Setting table holds product data the UI can change at runtime: budget cap, rubric definitions, competitor list, price tables, prompt versions. If a change should require a restart, it is an env var; if it should be a click, it is a Setting. This chapter enumerates both, with defaults and purpose, so that no code session ever has to grep for a knob.

Environment variables

Variable	Default	Purpose
SESSION_PASSPHRASE_HASH	(set at first run)	Argon2id hash of the local session passphrase (T4)
DATABASE_URL	file:./dev.db	SQLite file location (Prisma)
MEDIA_DIR	./data/media	Content-addressed audio vault
BACKUP_DIR	./data/backups	Snapshot target for the nightly job
PORT	3000	App port
LOG_LEVEL	info	debug, info, warn, error
ROUTING_MODE	LOCAL_FIRST	LOCAL_FIRST, CLOUD_FIRST, LOCAL_ONLY, CLOUD_ONLY
OLLAMA_BASE_URL	http://127.0.0.1:11434	Local runtime endpoint
OLLAMA_TIMEOUT_MS	120000	Per-call local timeout (large contexts are slow)
OPENAI_API_KEY / OPENAI_BASE_URL	(empty) / api.openai.com	OpenAI or any compatible relay
ANTHROPIC_API_KEY	(empty)	Claude fallback leg
GEMINI_API_KEY	(empty)	Gemini fallback leg
COMPATIBLE_BASE_URL / COMPATIBLE_API_KEY	(empty)	The any-provider escape hatch (Groq, DeepSeek, Together...)
WHISPER_MODEL	small	small, medium, large-v3 - tier ladder per Ch. 10
WHISPER_DEVICE	cpu	cpu or cuda; auto-detects and logs the choice
WHISPER_COMPUTE	int8	Quantization for CPU comfort (R05)
AUDIO_MAX_MB / AUDIO_MAX_HOURS	500 / 3	Upload rules (FR-001)
EMBED_MODEL	nomic-embed-text	Local embedding model tag
MONTHLY_BUDGET_USD	5.00	Cloud cap seed; runtime value lives in Setting budget.monthlyCapUsd
EVAL_THRESHOLD_OVERRIDE	(empty)	Per-task threshold overrides for experiments only
JOB_MAX_ATTEMPTS	3	Retry ceiling before parking (Table 3.2)
WORKER_ENABLED	true	Set false on a UI-only machine (rare, but free)
SSE_HEARTBEAT_SEC	15	Keepalive cadence (Table 5.8)

Table 11.1 - Every environment variable (defaults shown for a fresh install)

models.yaml: the model catalog

The catalog resolves the model hints from Chapter 6 into concrete, pullable model ids and defines the preference chain per hint. It is the single place a model swap happens (R08), and provider health plus the catalog are what the Settings screen renders. The seeded file below is the 2026-October baseline validated in v1; entries are self-describing, and unknown providers simply never become healthy.

```
# models.yaml (seeded; edit freely - this is data, not code)
catalog:
  small:            # hint -> concrete models, preference order
    - provider: ollama
      model: qwen3:8b
      contextTokens: 32768
  medium:
    - provider: ollama
      model: gpt-oss:20b
      contextTokens: 32768
    - provider: openai          # first cloud leg (needs key)
      model: gpt-4o-mini
      contextTokens: 128000
      pricePerMTokens: { input: 0.15, output: 0.60 }   # USD, maintained in Setting
    - provider: anthropic
      model: claude-sonnet (current stable tag)
      contextTokens: 200000
    - provider: gemini
      model: gemini-flash (current stable tag)
      contextTokens: 1000000
  large: []             # unused in v1; reserved for RAG-heavy future
  embed:
    - provider: ollama
      model: nomic-embed-text
      dim: 768
overrides: {}           # per-task chains, e.g. { T04: { only: [ollama] } }
retention:              # provider data-policy flags (T7)
  openai: zero-retention-api
  anthropic: zero-retention-api
  gemini: free-tier-stores
  compatible: unknown
```

models.yaml seeded baseline. Preference order within a hint is top-to-bottom; the router skips unhealthy or keyless entries.

Setting keys and tuning knobs

Setting keys are whitelisted by PATCH /api/settings - the whitelist itself is code, so a new key is a deliberate act with a schema. The defaults below are the seeded values; every one of them is chosen so that the app behaves correctly without ever opening Settings. Tuning guidance follows the table for the three knobs people actually adjust.

Key	Seeded value	Purpose
budget.monthlyCapUsd	5.00	The cap the ledger enforces (FR-022); changes are logged
rubric.v1	Table 3.5 definition	Rubric data: dimensions, weights, full-marks text; versioned immutably
competitors.list	["Salesforce", "HubSpot", "Zoho", "Pipedrive"]	Seed list for COMPETITOR_MENTIONED; edit freely
prices.openai / prices.anthropic / prices.gemini	2026 mid-tier table	Per-model USD per million tokens; keeps ledger estimates honest
ui.tone	warm-professional	Default tone for T07 drafts
redaction.nerEnabled	true	Local NER pass on top of regex rules
transcript.readerFont	serif	serif or dyslexia-friendly (Ch. 7)
search.minScore	0.24	Cosine floor below which hits are hidden (tuned on fixtures)
pipeline.autoArchiveDays	0	0 = never auto-archive; else days after REVIEWED

Table 11.2 - Setting keys (runtime, UI-editable)

The three knobs worth tuning guidance: ROUTING_MODE is the privacy-cost dial - LOCAL_ONLY when a spike of paranoia or an offline week hits, LOCAL_FIRST as the designed default, CLOUD_FIRST only if local quality measurably disappoints on your golden set (check the eval console first, not vibes). MONTHLY_BUDGET_USD exists to make cloud anxiety numeric: at default caps a heavy month of escalated analysis costs about a coffee. WHISPER_MODEL is the patience dial: small is the default because its RTF holds on CPU; medium buys accuracy on accented audio (G5 is the fixture that decides) at roughly double the wait. Every other knob keeps its default until a measurement in the eval console or performance suite says otherwise - that is the standing rule of this dossier.

12 · Build Orders - The Execution Plan

This chapter converts everything before it into forty work orders across six phases, P0 through P5. A work order is the unit of a working session: it names its scope, the key files it touches, its dependencies, its estimate in 90-minute focus blocks, and a definition of done that cites requirement IDs - the same FRs and NFRs from Chapter 2. The rules of engagement are three. Build in order within a phase (dependencies are listed, not implied). A work order is done when its DoD passes, not when the code exists. And when reality disagrees with this dossier mid-build, the disagreement is resolved by the change-control rules at the end of this chapter - never by quiet improvisation, which is how specifications rot.

Phase 0 - Foundation

Phase 0 exists to make every later session friction-free: scaffold, schema, tokens, and the trust layer. It is deliberately infrastructure-light (no AI, no audio) and ends with an app you can sign into and look at. Six work orders, eleven blocks.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-001	Repo scaffold: Next.js 16 + TS strict, pnpm, lint/format, CI script, .env.example	root, package.json	-	1	pnpm dev serves an empty shell; lint + unit runner wired; commit hooks active
WO-002	Prisma schema per Ch. 4 (all 19 tables + enums + indexes) and first migration	prisma/schema.prisma, migrations/	001	2	migrate deploy on clean clone creates dev.db; seed runs idempotently with default Settings and starter prompt templates
WO-003	Warm Calm tokens + component kit + app shell (left rail, header, toast, drawer)	globals.css, components/ui/*	001	2	Tokens from Table 7.1 exist once; shell renders on /; storybook-style preview page shows every component in its four states
WO-004	Auth: passphrase session cookie + middleware + login screen	middleware, /api/auth/*, app/login	002, 003	1	NFR: unauthenticated /api returns 401; cookie flags per T4; login flow E2E-green
WO-005	Settings API + Settings screens skeleton (Providers/Budget/Redaction/Prompts tabs)	/api/settings, app/settings/*	002, 003	2	Whitelisted keys PATCH round-trips; all four tabs render their empty states per Ch. 7
WO-006	Job queue + worker loop with locks, backoff, and boot-time reclaim	lib/jobs/*	002	3	Unit tests: enqueue, lease, stale-lock reclaim, attempts ceiling; FR-008 stage-level assertions run against a fake job

Table 12.1 - Phase 0 work orders

Phase 1 - Ingestion and transcription

Phase 1 builds the left half of the pipeline: from a dropped file to a speaker-attributed transcript you can read. It ends with the first genuinely magical moment of the project - your own recording, transcribed, on your own machine. Six work orders, fourteen blocks.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-007	Media vault: SHA-256 content addressing, two-level fan-out storage, dedup	lib/media/*	002	2	FR-002 integration-green (duplicate links, no re-job); corrupt-file case handled per Table 3.6
WO-008	Upload API + Inbox screen + dropzone + upload rules	/api/calls, app/inbox, app/upload	007	2	FR-001 table-driven tests pass; 415 envelope correct; inbox renders statuses with progress rail
WO-009	Normalize stage: ffmpeg strip/resample/loudness + ffprobe facts	pipeline/normalize*	006, 007	1	WAV asset written; duration/sampleRate persisted; NORMALIZE_FAILED path tested
WO-010	Transcribe stage: whisper bindings, model ladder, RTF logging	pipeline/transcribe*	009	3	FR-003 measured on reference machine (RTF logged per run); G1-G3 transcripts produced; TRANSCRIBE_EMPTY path tested
WO-011	Diarization + speaker relabel (T02) with local LLM pass	pipeline/diarize*, prompts/T02	010	2	Labels REP/PROSPECT/UNKNOWN on G1-G4; relabel provenance row written; manual label fix on reader persists
WO-012	Transcript reader screen + in-call find + selection popover	app/calls/[id]/transcript	011	4	Reader renders serif, speaker rail, ms-accurate seek; popover offers promote-to-note; keyboard layer for reader green

Table 12.2 - Phase 1 work orders

Phase 2 - Analysis engine

Phase 2 is the heart transplant: the provider adapter family, the routing function, the prompt engine, and tasks T03-T08 turning transcripts into intelligence. It ends with a call detail page that explains itself. Nine work orders, twenty blocks - the longest phase, split by design so every order still fits a session.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-013	Adapter family: uniform interface, Ollama + OpenAI + Anthropic + Gemini + compatible, health caching	lib/providers/*	002	3	NFR-08: each at or under 150 LOC; mock-server integration tests; health card endpoint live
WO-014	Routing function: policy tier, payload fit, budget check, health skip, preference order	lib/routing/*	013	2	Unit tests cover every row of Table 6.4; pure function, zero I/O; property tests for cap refusal
WO-015	Budget ledger: transactional cap, entries, spend endpoints	lib/budget/*, /api/budget	002, 013	2	FR-022 integration-green with the 0.01-dollar cap trick; footer meter wired to SSE budget.warning
WO-016	Prompt engine: template store, versioning, activation gate hook	lib/prompts/*, Settings-Prompts UI	002, 013	2	NFR-14: edit creates version; exactly one active; activation blocked when eval gate red
WO-017	Quality gates + repair-retry ladder shared by all tasks	lib/tasks/gates*	014, 016	2	Schema-validators per task from Ch. 6; retry ladder per Table 6.4 unit-green on failure fixtures
WO-018	Analysis stage: T03 summary, T05 signals, T08 notes with seeded prompts	pipeline/analyze*, prompts/T03,T05, T08	017	3	G1, G4, G7 outputs schema-valid with evidence ids; ProviderRun rows complete (FR-023)
WO-019	T04 MEDDIC extraction + write-back to deals/contacts	prompts/T04, lib/meddic*	018	2	FR-009 eval F1 at or above 0.85 on golden set; NOT_FOUND honest on G4; champion/EB write-back green
WO-020	T06 scorecard + S5 arithmetic metrics	prompts/T06, lib/score*	018	2	FR-014/005: weighted scoring unit-green; arithmetic fields never trusted from model (asserted); G2 scored with evidence
WO-021	T07 follow-up draft + notes/review UI on call detail	prompts/T07, app/calls/[id]/*	018	3	FR-006/007: draft cites concrete detail on G1; edit + copy + SENT flow E2E-green

Table 12.3 - Phase 2 work orders

Phase 3 - Deal Intelligence

Phase 3 gives the intelligence a home: deals, contacts, the board, the timeline, and health. Five work orders, twelve blocks, ending with the pipeline board reading like the truth.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-022	Deal + Contact CRUD APIs and screens	/api/deals, /api/contacts, app/deals/*	005, 008	2	Soft-delete per Ch. 4; contacts deal-scoped; validation envelopes per Table 5.2
WO-023	Pipeline board: columns, cards, badges, drag with optimistic update	app/board, /api/pipeline	022	3	FR-011 E2E-green incl. drag ghost and rollback on error; board under 1 s with 50 seeded deals
WO-024	Deal timeline assembly + risk badge surfacing	lib/timeline/*, app/deals/[id]	022, 018	2	FR-012: mixed entries ordered correctly; every entry links to source; risk badges match flag rules
WO-025	Health score service (v1 formula) + recomputation hooks	lib/health/*	024	1	Table 3 formula unit-pinned; recompute on analyze verified; historical scores explainable
WO-026	Insight review queue + confirm/dismiss flows with write-back	app/review, /api/insights	019	4	FR review flow E2E; confirmed MEDDIC writes to deal/contact; queue keyboard layer green

Table 12.4 - Phase 3 work orders

Phase 4 - Coaching and Library

Phase 4 closes the four pillars: trends, best practices, embeddings, semantic search, and the RAG answer surface. Six work orders, fourteen blocks.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-027	Coaching home: trend charts, dimension bars, calibration empty state	app/coaching, /api/coaching/*	020	2	FR-015: chart renders only past 3 calls; window filter; table view toggle
WO-028	Scorecard detail + compare-to-top-quartile	app/calls/[id]/scorecard	020, 027	2	FR-017: ghost quartile markers; evidence links open reader at segment
WO-029	Embedding pipeline + index stage (T10) + re-index job	pipeline/index*, lib/embed/*	010, 013	2	Chunking per Ch. 3; re-index on model switch green; vectors removed with sources
WO-030	Library search API + results screen with snippets, filters, grouping	app/library, /api/library/*	029	3	FR-018/019: latency under 2 s at 1k synthetic calls; filters compose; highlight correct
WO-031	Best practices: promotion flow, candidate suggestions (T12), library CRUD	app/practices, prompts/T12	018, 030	2	FR-016: promote from selection and accept suggestions both create searchable entries with evidence
WO-032	Library answer surface (T11 RAG) with citations and refusal	app/library/answer, prompts/T11	030	3	Citation validity 1.00 on fixtures; refusal state on low-score; G9 adversarial case green

Table 12.5 - Phase 4 work orders

Phase 5 - Hardening and polish

Phase 5 earns the right to be called done: privacy enforcement, the eval console, backup, export, performance, and the operational dry-run that proves the runbook. Eight work orders, sixteen blocks. The phase ends by running the project definition of done below, in order, on the reference machine.

ID	Scope	Key files	Depends	Est.	Definition of done
WO-033	Redaction pipeline (T09) + Settings rules UI + test pad	lib/redact/*, app/settings/redaction	013	3	T9/T10 threat answers: G10 zero leaks; adapter-boundary spy test build-failing (NFR-05)
WO-034	Eval harness CLI + golden set wiring + eval console screen (Should tier)	lib/eval/*, app/settings/eval	016, 019, 020	3	npm run eval scores all tasks vs thresholds; EvalRun rows visible; promotion gate enforced in UI
WO-035	Provider test buttons + spend chart + budget warning SSE	app/settings/providers, /api/providers/*	015	1	FR-021/022 surfaces complete; unhealthy provider states render per Ch. 7
WO-036	Backup job + restore drill script + Data and backup screen	scripts/backup*, app/settings/data	007	2	NFR-10: backup in 2 commands; restore drill passes on scratch dir; snapshot list renders
WO-037	Export bundle (FR-024) with redaction scope ALL	lib/export/*, /api/system/export	033	1	Bundle imports into a fresh clone (reconstruction test); keys absent by schema
WO-038	Performance suite: RTF, p95, search-at-1k, startup	perf/*	030, 010	2	NFR-01/02/03/12 thresholds measured and logged; failures block phase acceptance
WO-039	E2E full-journey suite + SIGKILL crash drill at each stage	e2e/*	all prior	2	FR-008 proven per stage boundary; happy path under 5 minutes; nightly job scheduled
WO-040	Runbook dry-run + dogfood week + v2 backlog harvest	docs/runbook, CHANGELOG	all prior	2	Ch. 10 dry-run checklist executed and dated; 5 real calls processed end-to-end; friction notes filed to v2 list, not the plan

Table 12.6 - Phase 5 work orders

Project definition of done and change control

The project is done when this list passes in order, on the reference machine, from a clean clone: all forty work orders' DoDs green; every FR acceptance criterion demonstrated at least once in the E2E suite; the eval harness green on active prompts (Table 6.5 thresholds); the performance suite green (Table 2.6 NFR thresholds); the egress and crash-safety invariants green; the runbook dry-run executed with a dated checklist; and five real calls processed end-to-end from upload to archived. That final state is the answer to "when can I stop planning and call it built" - and it is fully reachable with the materials in this dossier.

Change	Allowed path	Re-plan needed?
Prompt wording, model ids, catalog chains	Settings/models.yaml + eval gate before activation	No
Rubric weights, competitor list, price tables, budget cap	Setting edit, versioned where history matters	No
Bug fixes that preserve FR acceptance criteria	Normal work order + regression test	No
New insight types, screens, or endpoints	v2 backlog list - explicitly out of current plan	Only when v2 starts
Changes to an FR's acceptance criterion	Change request against Ch. 2, eval impact assessed first	Yes - this dossier
Schema changes beyond additive	Ch. 4 migration policy (add, backfill, later remove)	No, if policy followed
Architecture reversals (monolith, SQLite, adapters)	Stop; write v3 blueprint	Yes - a new volume

Table 12.7 - Change control: what can change without re-planning

Forty work orders, one hundred and one focus blocks, six phases, one builder. The plan is complete: every artifact it needs exists in these twelve chapters, every requirement carries its test, every risk has its mitigation, and every session between here and "built" has a name and a definition of done. The next action on this project is not another document - it is WO-001.